

Heaven's Light is Our Guide



**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
RAJSHAHI UNIVERSITY OF ENGINEERING & TECHNOLOGY,
BANGLADESH**

**QFlow: A Fixed-Point FPGA Accelerator for Adaptive
Key-Resource-Aware Route-Candidate Evaluation in
Trusted-Relay QKD Networks**

A thesis submitted in partial fulfillment of the requirements
for the degree of Bachelor of Science in Computer Science & Engineering

Author

Shahriar Rizvi

Roll No. 2003104, Section B, Series 2020

Supervised by

Dr. Md. Nazrul Islam Mondal

Professor

Department of Computer Science & Engineering

Rajshahi University of Engineering & Technology

August 2026

ACKNOWLEDGEMENT

All praise and gratitude are due to Almighty Allah for granting me the strength, patience, and opportunity to complete this research.

I express my sincere gratitude to my supervisor, **Dr. Md. Nazrul Islam Mondal**, Professor, Department of Computer Science & Engineering, Rajshahi University of Engineering & Technology. His guidance, careful questions, and encouragement shaped the research objectives and helped me connect the mathematical design with reproducible hardware experiments.

I am thankful to the faculty and staff of the Department of Computer Science & Engineering, RUET, for the academic foundation and research environment that made this work possible. I also acknowledge the open-source communities behind Python, Icarus Verilog, Yosys, OpenROAD, and ngspice, as well as the researchers whose published work provided the technical context for this study.

Finally, I offer heartfelt thanks to my parents, family, friends, and classmates for their patience, prayers, and constant support throughout my undergraduate study.

Heaven's Light is Our Guide



**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
RAJSHAHI UNIVERSITY OF ENGINEERING & TECHNOLOGY,
BANGLADESH**

CERTIFICATE

*This is to certify that the thesis entitled “**QFlow: A Fixed-Point FPGA Accelerator for Adaptive Key-Resource-Aware Route-Candidate Evaluation in Trusted-Relay QKD Networks**”, submitted by **Shahriar Rizvi, Roll No. 2003104**, in partial fulfillment of the requirements for the award of the degree of Bachelor of Science in Computer Science & Engineering at Rajshahi University of Engineering & Technology, is a record of the candidate's own work carried out under my supervision. To the best of my knowledge, this work has not been submitted, in whole or in part, for the award of any other degree or diploma.*

Supervisor

External Examiner

Dr. Md. Nazrul Islam Mondal

Professor

Department of Computer Science & Engineering

Rajshahi University of Engineering &
Technology

Rajshahi-6204, Bangladesh

External Examiner

Department of Computer Science & Engineering

Rajshahi University of Engineering &
Technology

Rajshahi-6204, Bangladesh

DECLARATION

I hereby declare that the work presented in this thesis is original and was conducted by me under the supervision of **Dr. Md. Nazrul Islam Mondal**, Professor, Department of Computer Science & Engineering, Rajshahi University of Engineering & Technology. This thesis has not been submitted, in whole or in part, to this or any other institution for the award of another degree or diploma.

All external ideas, publications, software tools, and data sources used in this work have been acknowledged. Physical measurements, post-route implementation results, RTL and software simulations, analytical calculations, and supporting physical-design results are identified according to their respective evidence types.

The repository materials, scripts, fixed seeds, model records, verification vectors, implementation reports, and physical replay summaries are maintained to support reproducibility and independent examination of the reported results.

Date: August 2026

Place: RUET, Rajshahi

Shahriar Rizvi

Roll No. 2003104

ABSTRACT

Quantum key distribution (QKD) networks require control decisions that account for the availability and operating condition of link key resources. Most published QKD routing studies evaluate network-level algorithms in software, while FPGA research has mainly accelerated physical-layer or post-processing functions. This thesis presents **QFlow**, a deterministic fixed-point FPGA accelerator that evaluates externally generated route candidates for trusted-relay QKD networks. Candidate generation, topology management, key-management-system operation, software-defined-network orchestration, security-policy enforcement, and route installation remain control-plane functions outside the accelerator.

The controller quantizes four features into radix-4 values, producing 256 states and four scoring-profile actions. Tabular Q-learning is performed offline. The selected policy is deployed as a 256-entry, two-bit read-only memory with minimum-dwell control; FPGA execution performs deterministic inference and does not update Q-values. Five independent hardware evaluation configurations, H0–H4, implement a feasible shortest-distance baseline, a fixed key-aware baseline, a fixed-profile **QFlow** evaluator, a threshold/rule-adaptive evaluator, and a reinforcement-learned adaptive evaluator.

Verification combined a fixed-point software reference, exhaustive state and threshold tests, cycle-accurate register-transfer-level simulation, independent Xilinx ISE implementations, and physical replay on a Digilent Nexys 3 Spartan-6 FPGA. The physical campaign produced 420 records containing 3,780 checked fields, with zero mismatches. All five configurations exceeded 100 MHz after placement and routing. Configuration H4 achieved a post-route maximum frequency of 102.020 MHz and a mean normal-case accepted-request-to-done kernel latency of 2.048 μ s. Relative to configuration H3, configuration H4 changed mean latency by +0.986%, maximum frequency by -0.612% , look-up-table use by -7.081% , register use by +5.378%, and occupied-slice use by +4.961%. The complete H4 controller, comprising the learned policy and dwell mechanism, produced 45.455% fewer profile transitions than H3 on the evaluated common replay. Held-out route-quality differences were not statistically significant, so the evidence establishes physical deployability and distinct adaptive behavior rather than route-quality superiority.

Supporting synthesis and OpenROAD experiments demonstrate implementation depth for selected arithmetic and selection kernels and indicate a possible future ASIC path; they are not an integrated ASIC implementation. The results demonstrate that an offline-learned key-resource-aware controller can be deployed bit-exactly on a resource-constrained FPGA with deterministic microsecond-scale candidate-evaluation latency and controlled implementation overhead.

Keywords: quantum key distribution network, FPGA accelerator, route-candidate evaluation,

fixed-point arithmetic, reinforcement learning, policy ROM, trusted relay.

CONTENTS

ACKNOWLEDGEMENT	ii
CERTIFICATE	iii
DECLARATION	iv
ABSTRACT	v
	Pages
LIST OF TABLES	xi
LIST OF FIGURES	xiii
LIST OF ABBREVIATIONS	xiv
LIST OF SYMBOLS	xvi
1 Introduction	1
1.1 Introduction	1
1.2 Motivation	1
1.2.1 QKD links and trusted-relay networking	1
1.2.2 Key-pool-aware routing control	2
1.2.3 Route-candidate evaluation as the selected problem	2
1.2.4 Why deterministic hardware acceleration is relevant	2
1.3 Research Gap	3
1.4 System Scope and Boundary	3
1.5 Problem Statement	3
1.6 Research Objectives	4
1.7 Research Questions	4
1.8 Research Hypotheses	4
1.9 Contributions	5
1.10 Practical Significance	5
1.11 User Requirements and Broader Considerations	6
1.12 Thesis Organization	6

1.13 Conclusion	6
2 Background and Literature Review	7
2.1 Introduction	7
2.2 Trusted-Relay QKD Networks	7
2.3 Key Pools and Resource Evolution	7
2.4 Hard Eligibility and Soft Preference	8
2.5 Candidate-Route Evaluation	8
2.6 Multi-Objective Scoring	8
2.7 Offline Reinforcement Learning for Profile Selection	9
2.8 Fixed-Point Arithmetic and FPGA Implementation	9
2.9 CPU, FPGA, and ASIC Characteristics	10
2.10 Literature Review	10
2.10.1 QKD routing and network control	10
2.10.2 Operational QKD systems	11
2.10.3 FPGA acceleration in QKD	11
2.10.4 Comparison taxonomy	11
2.10.5 Reproducible literature search and verification	12
2.10.6 Research position	12
2.11 Data-Analysis Principles	13
2.12 Conclusion	13
3 Methodology	14
3.1 Introduction	14
3.2 System Overview	14
3.3 System Boundary	14
3.4 Network and Key-Resource Model	14
3.5 Candidate-Route Representation	15
3.6 Eligibility Rules	15
3.7 Fixed-Point Candidate-Scoring Model	16
3.8 Fixed-Point Scoring-Profile Codebook	16
3.9 Hardware Evaluation Configurations H0–H4	17
3.10 Controller State Representation	18
3.11 Offline Policy Learning	19
3.12 Policy and Profile ROM Deployment	20
3.13 Minimum-Dwell Mechanism	20
3.14 RTL Architecture	20
3.15 Fixed-Point Representation	21
3.16 Experimental Inputs, Workloads, and Trace Partitions	21

3.17	Verification Methodology	21
3.18	FPGA Implementation	22
3.19	Physical Replay Protocol	22
3.20	Statistical Analysis	23
3.21	Supporting VLSI Methodology	23
3.22	Methodological Limitations	23
3.23	Conclusion	24
4	Results and Discussion	25
4.1	Introduction	25
4.2	Experimental Overview	25
4.3	Offline Controller Behavior	25
4.4	Controller and Policy Verification	26
4.5	Integrated RTL Verification	26
4.6	Independent FPGA Implementation	26
4.7	Same-Board Physical Replay and Exactness	27
4.8	Post-Route Timing and Kernel Latency	27
4.9	Resource Utilization	28
4.10	Adaptive Profile Behavior	29
4.11	Held-Out Route-Quality Analysis	29
4.12	H4 Versus H3 Engineering Trade-Off	30
4.13	Comparison with Related Work	30
4.14	Practical Deployment Scenario	31
4.15	Platform Selection and Cost Implications	32
4.15.1	CPU role	32
4.15.2	FPGA role	32
4.15.3	ASIC role	32
4.15.4	Conceptual total-cost model	32
4.16	Kernel-Level Physical-Design Exploration	33
4.17	Threats to Validity	34
4.18	Answers to the Research Questions	35
4.19	Research-Hypothesis Outcomes	35
4.20	Conclusion	36
5	Conclusion and Future Work	37
5.1	Thesis Summary	37
5.2	Main Findings	37
5.3	Contributions	38
5.4	Limitations	38

5.5	Future Work	38
5.6	Closing Statement	39
A	Reproducibility and Artifact Register	40
A.1	Repository snapshots	40
A.2	Primary artifact register	40
A.3	Headline numerical sources	40
A.4	Evidence classification	42
A.5	Tool versions	42
A.6	Build and replay sequence	42
B	Fixed-Point, State, and Protocol Contract	44
B.1	State and policy layout	44
B.2	Threshold definitions	44
B.3	Profile payload	44
B.4	Request and result protocol	45
B.5	Latency boundary	45
C	Complete Additional Results	46
C.1	Additional Same-Board Route-Quality Results	46
C.2	Adaptive Profile Percentages	46
C.3	Controller Overhead Relative to Configuration H0	46
C.4	Integration Timing Inventory	47
C.5	Supporting VLSI Detail	47
D	Experimental Scope and Reproducibility Notes	49
D.1	Measurement boundaries	49
D.2	Repository milestone identifiers	49
D.3	Reproducibility records	49
	REFERENCES	50

LIST OF TABLES

No.	Table Name	Page
2.1	Comparison classes used throughout the thesis.	11
2.2	Representative related-work positioning. Direct numerical comparability requires an equivalent function and measurement boundary.	12
3.1	Fixed-point scoring-profile codebook.	17
3.2	Hardware evaluation configurations H0–H4.	18
3.3	Experimental inputs and trace partitions.	21
4.1	Software-simulated held-out controller behavior over four 512-decision traces.	25
4.2	Controller and policy verification summary.	26
4.3	Physical replay exactness for the five independent configurations.	27
4.4	Post-route maximum frequency and measured kernel latency under the common 100 MHz operating contract.	27
4.5	Post-route resource utilization of independent H0–H4 implementations.	28
4.6	Profile use and switching over 81 valid adaptive decisions.	29
4.7	Paired held-out physical route-quality comparisons.	29
4.8	Configuration H4 relative to rule-adaptive configuration H3.	30
4.9	Function- and evidence-aware comparison with representative related work.	31
4.10	Qualitative platform comparison for the QFlow development stages.	33
4.11	OpenROAD physical-design results for isolated kernels.	34
4.12	Answers to the research questions.	35
4.13	Mapping from research hypotheses to evidence and outcomes.	35
A.1	Primary reproducibility artifacts.	40
A.2	Authoritative sources for headline numerical results.	41
A.3	Evidence types used in the thesis.	42
B.1	Adaptive-controller storage layout.	44
B.2	Synchronous kernel protocol.	45
C.1	Held-out status and route-quality summaries.	46
C.2	Adaptive profile use over 81 valid decisions.	46

C.3	Absolute resource and cycle differences relative to minimal configuration H0.	46
C.4	Post-route timing checkpoints for successively integrated blocks.	47
C.5	Generic Yosys synthesis results for isolated kernel variants.	47
C.6	Simplified ngspice transmission-gate multiplexer load sweep.	48

LIST OF FIGURES

No.	Figure Name	Page
1.1	System context of QFlow . Candidate generation and route installation are control-plane functions outside the FPGA kernel.	1
3.1	End-to-end methodology and the FPGA kernel boundary.	14
3.2	Four-feature state encoding and policy-ROM inference in configuration H4.	19
3.3	Common candidate-evaluation RTL architecture.	20
4.1	Timing and latency across the H0–H4 comparison.	28
4.2	Trace-specific adaptive behavior of configurations H3 and H4.	29
4.3	Routed OpenROAD layouts for three isolated kernels. These views do not represent an integrated QFlow ASIC.	34

LIST OF ABBREVIATIONS

Abbreviation	Meaning
ASIC	Application-Specific Integrated Circuit
BRAM	Block Random-Access Memory
CI	Confidence Interval
DRC	Design Rule Check
DRL	Deep Reinforcement Learning
DSP	Digital Signal Processing slice
FDPE	Fidelity Decay Prediction Engine (repository kernel identifier)
FPGA	Field-Programmable Gate Array
FSM	Finite-State Machine
GA	Genetic Algorithm
GDS	Graphic Data System layout stream
H0–H4	Hardware evaluation configurations
ISE	Xilinx Integrated Software Environment
KMS	Key Management System
LUT	Look-Up Table / FPGA logic element, according to context
LVS	Layout Versus Schematic
MMIO	Memory-Mapped Input/Output
OMNeT++	Objective Modular Network Testbed in C++
PAR	Place and Route
PMO-GA	Pipelined Multi-Objective Genetic Algorithm
QBER	Quantum Bit Error Rate
QKD	Quantum Key Distribution
RL	Reinforcement Learning
ROM	Read-Only Memory
RTL	Register-Transfer Level
SDN	Software-Defined Networking
SKAG	Stochastic Key Availability Graph (repository kernel identifier)
SoC	System on Chip
UART	Universal Asynchronous Receiver/Transmitter
WNS	Worst Negative Slack

Abbreviation	Meaning
XST	Xilinx Synthesis Technology

LIST OF SYMBOLS

Symbol	Meaning
$G = (V, E)$	Directed trusted-relay QKD network graph
$K_{ij}(t)$	Available classical key units in link pool (i, j) at time t
$K_{ij}^{\max}$	Configured capacity of key pool (i, j)
$\lambda_{ij}(t)$	Effective key-generation rate of link (i, j)
$C_{ij}(t)$	Key-consumption rate of link (i, j)
$X_{ij}(t)$	Key units expired or revoked by policy
$o_{ij}(t)$	Normalized occupancy of key pool (i, j)
$q_{ij}(t)$	Externally measured QBER or health indicator for link (i, j)
α_{fiber}	Physical fibre attenuation coefficient
$\mathcal{R}_{sd}$	Externally generated candidate set for source s and destination d
$\mathcal{R}_i$	Candidate route with index i
$\alpha^{(k)}$	Scoring-coefficient vector for profile Π_k
$\lambda_{\text{TCH}}^{(k)}$	Tchebycheff objective-weight vector for profile Π_k
Π_k	Scoring profile k , where $k \in \{0, 1, 2, 3\}$
s_i	Fixed-point scalar score of candidate $\mathcal{R}_i$
$\mathbf{x}[n]$	Four-feature controller observation at decision epoch n
$z[n]$	Encoded controller state in $\{0, \dots, 255\}$
$a[n]$	Selected profile action in $\{0, 1, 2, 3\}$
$\pi(z)$	Offline-trained policy-ROM mapping from state to profile
$d_{\min}$	Minimum profile dwell in accepted decisions
T_{clk}	FPGA clock period
L	Kernel latency in cycles or time
$f_{\max}$	Post-route maximum operating frequency

CHAPTER 1

INTRODUCTION

1.1 Introduction

Trusted-relay quantum key distribution (QKD) networks must coordinate quantum-link resources with conventional network control. This thesis addresses one recurring control-plane computation: evaluating a bounded set of admissible route candidates from key-resource and network-state metadata. It develops **QFlow**, a deterministic fixed-point FPGA accelerator, and evaluates five independently implemented hardware configurations under a common physical measurement contract.

1.2 Motivation

1.2.1 QKD links and trusted-relay networking

QKD enables two endpoints to establish shared secret material by combining quantum transmission with authenticated classical communication. After sifting, parameter estimation, error correction, and privacy amplification, the usable keys are classical data managed by a key management system (KMS). A network service therefore depends not only on the quantum link but also on key storage, authorization, consumption, replenishment, monitoring, and failure handling [1, 2].

Direct QKD links do not necessarily connect every communicating pair. Trusted-relay networks extend coverage by forwarding key material or coordinating secure segments through intermediate trusted sites. Operational demonstrations such as MadQCI and a carrier-scale network show the importance of integrating QKD devices with KMS and software-defined networking (SDN) functions [3, 4]. Route selection becomes a control problem because candidate paths can differ in distance, hop count, available key material, key-generation rate, utilization, and operational health.

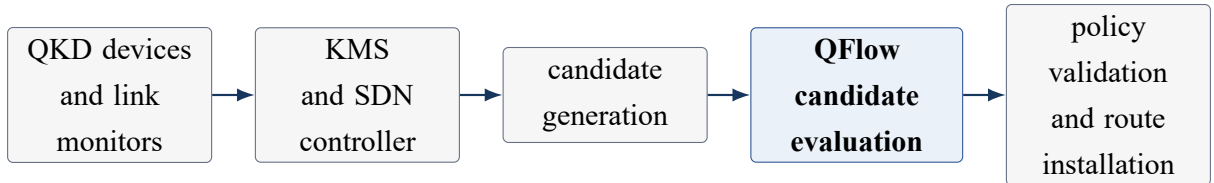


Figure 1.1. System context of **QFlow**. Candidate generation and route installation are control-plane functions outside the FPGA kernel.

1.2.2 Key-pool-aware routing control

Post-processed keys are finite classical resources. For a directed link, the KMS can report pool occupancy, effective key-generation rate, consumption, link status, utilization, and externally measured health indicators such as QBER-derived status. An optional age or expiry field represents KMS or security policy; it is not a quantum coherence property of stored bits. A route that is short in distance may be undesirable when one of its key pools is depleted, while a slightly longer route may provide greater reserve or replenishment capacity. Recent work consequently studies dynamic key-aware, multipath, joint-resource, and reinforcement-learning approaches [5, 6, 7, 8, 9].

Two decision layers are useful. Hard eligibility first excludes a route whose links are unavailable, under-provisioned, or administratively invalid. Soft preference then ranks the remaining routes according to a selected scoring profile. Keeping these layers separate prevents a high score from making an inadmissible route appear usable.

1.2.3 Route-candidate evaluation as the selected problem

Full graph routing includes topology acquisition, constraint processing, candidate generation, global optimization, security validation, and route installation. Mapping that entire workflow to a small FPGA would mix irregular control tasks with a compact recurring arithmetic task. This thesis instead accelerates the latter. For source s and destination d , software supplies

$$\mathcal{R}_{sd} = \{\mathcal{R}_0, \mathcal{R}_1, \dots, \mathcal{R}_{m-1}\}. \quad (1.1)$$

Each candidate $\mathcal{R}_i$ has a fixed-width descriptor containing feasibility, distance or consumption cost, key-resource conditions, load-related indicators, and a deterministic identifier. **QFlow** selects a scoring profile, evaluates the descriptors, and returns the chosen identifier, score, profile, and status.

This boundary makes exact verification possible. The same descriptor and fixed-point contract can be evaluated in software, RTL simulation, and physical hardware. It also reflects a practical heterogeneous design in which the CPU retains graph and policy responsibilities while the FPGA offloads repeated deterministic evaluation.

1.2.4 Why deterministic hardware acceleration is relevant

A candidate evaluator applies the same bounded operations to every request: quantization, read-only policy lookup, weighted fixed-point scoring, feasibility checks, comparison, and deterministic tie-breaking. These operations are suitable for dedicated pipelines and finite-state control. FPGA implementation provides cycle-level predictability, explicit numeric widths, and reconfiguration as the controller or scoring profiles evolve.

The present research stage also benefits from avoiding ASIC fabrication and from implementing all comparison configurations on the same device. CPU software remains essential for orchestration, candidate generation, offline learning, and irregular tasks. The platform choice is therefore a partitioning decision rather than a claim that one platform is universally superior.

1.3 Research Gap

The literature contains sophisticated software studies of QKD routing and resource allocation, including adaptive and reinforcement-learning methods. Separately, FPGA-based QKD research has demonstrated high-throughput reconciliation, post-processing, and synchronization [10, 11, 12, 13]. Within the documented literature audit, no directly comparable same-board implementation of the evaluated candidate-selection function was identified. The gap is therefore an implementation and evidence gap: a compact, adaptive, fixed-point candidate evaluator has not been compared through independent physical configurations with a common interface, exact reference agreement, and post-route timing and resource evidence.

1.4 System Scope and Boundary

QFlow is a route-candidate evaluator. It consumes candidate descriptors and network-state metadata that have already been validated and quantized at the software/hardware boundary. It supports fixed, rule-adaptive, and offline-learned profile selection. The learned configuration performs online inference from a read-only policy; Q-values are not updated during FPGA execution.

Global topology management, candidate generation, KMS operation, SDN orchestration, security-policy enforcement, secret-key delivery, and route installation remain in software or the control plane. The FPGA is neither a complete graph router nor a QKD physical-layer device. The physical experiment measures the accepted-request-to-done kernel boundary; it does not measure complete network provisioning latency.

1.5 Problem Statement

Given an externally generated candidate set $\mathcal{R}_{sd}$ and quantized key-resource and network-state metadata, the research problem is to design a bounded hardware kernel that:

1. rejects infeasible candidates according to explicit hard constraints;
2. selects a fixed or adaptive scoring profile deterministically;
3. evaluates feasible candidates using reproducible fixed-point arithmetic;
4. returns a stable result under a synchronous request/done protocol; and

5. is physically implementable on a resource-constrained FPGA with measurable timing, resource, exactness, and adaptive-behavior trade-offs.

1.6 Research Objectives

The objectives are to:

1. formulate a classical key-resource model based on pool occupancy, generation rate, consumption, status, utilization, health indicators, and policy-controlled freshness;
2. define a fixed-point candidate-scoring contract with distinct hard eligibility and soft preference;
3. develop fixed, rule-adaptive, and offline-learned profile controllers;
4. implement five independent FPGA configurations under one external contract;
5. verify reference-to-RTL and reference-to-board exactness;
6. measure post-route timing, kernel latency, resources, and profile behavior; and
7. interpret route-quality and platform trade-offs without crossing evidence boundaries.

1.7 Research Questions

- RQ1.** Can all five independently implemented configurations meet a 100 MHz post-route target on the selected FPGA?
- RQ2.** Do physical outputs match the fixed-point reference exactly under a common replay and output schema?
- RQ3.** What latency, maximum-frequency, and resource trade-offs arise across the fixed and adaptive configurations?
- RQ4.** Does the offline-learned controller exhibit distinct and controlled profile behavior relative to the rule-adaptive controller?
- RQ5.** Do the held-out physical comparisons establish statistically significant route-quality improvement for the learned configuration?

1.8 Research Hypotheses

- RH1.** Each independent hardware evaluation configuration will meet the 100 MHz post-route timing target.
- RH2.** Every checked physical result field will match the frozen fixed-point reference.

- RH3.** The reinforcement-learned adaptive configuration will retain microsecond-scale kernel latency with controlled implementation overhead.
- RH4.** The reinforcement-learned adaptive configuration with minimum-dwell control will produce fewer profile transitions than the rule-adaptive configuration on the common replay.
- RH5.** The reinforcement-learned adaptive configuration will achieve statistically significant route-quality improvement over fixed-profile and rule-adaptive alternatives on the held-out physical pairs.

1.9 Contributions

The thesis makes the following contributions:

1. a classical key-resource model for candidate evaluation based on pool occupancy, generation rate, consumption, operational status, policy freshness, and externally measured health indicators;
2. a deterministic fixed-point candidate-evaluation contract that separates hard eligibility from profile-conditioned multi-objective preference;
3. an adaptive controller with four radix-4 features, 256 states, four profile actions, offline tabular Q-learning, a 256-entry two-bit policy ROM, and minimum-dwell control;
4. five independent FPGA implementations with a common interface and measurement boundary;
5. a same-board campaign comprising 420 physical records, 3,780 checked fields, and exact reference agreement; and
6. a complete engineering comparison of timing, latency, resources, adaptive behavior, route-quality statistics, and stage-specific platform implications.

1.10 Practical Significance

In a prospective KMS/SDN deployment, candidate evaluation may recur whenever requests arrive or resource state changes. Offloading this fixed-width calculation can provide predictable control latency and a bit-exact implementation shared across tests and deployments. Reprogrammable policy and profile storage also allows the accelerator to evolve without replacing the surrounding control plane.

1.11 User Requirements and Broader Considerations

The engineering users of the proposed kernel are network-control and security software developers. Their principal requirements are deterministic requests and responses, explicit numeric formats, visible failure status, reproducible profiles, and a software fallback. A deployment must authenticate the host/FPGA interface, validate inputs, protect policy artifacts, log configuration identity, and retain the controller’s authority over final route installation.

The work does not process human-subject data. Its societal relevance lies in supporting reproducible research on secure-network control rather than promising a complete commercial service. Hardware safety is limited to normal laboratory electrical and electrostatic-discharge practice. Environmental benefit is not quantified because no activity-calibrated power measurement or life-cycle study was performed; reconfigurability can, however, extend prototype usefulness by allowing new bitstreams on the same device.

1.12 Thesis Organization

Chapter 2 reviews trusted-relay QKD networking, key-resource control, multi-objective scoring, reinforcement learning, fixed-point arithmetic, platform alternatives, and related work. Chapter 3 defines the system model, scoring profiles, hardware configurations, controller, RTL, verification, FPGA experiment, statistical method, and supporting VLSI method. Chapter 4 presents and discusses the software, RTL, post-route, physical replay, adaptive-behavior, route-quality, platform, and VLSI results. Chapter 5 summarizes the findings, contributions, limitations, and future work. The appendices provide reproducibility records, fixed-point and protocol details, additional results, and experimental-scope notes.

1.13 Conclusion

The selected problem is a bounded, recurring route-candidate evaluation task within a larger KMS/SDN workflow. The research therefore concentrates on fixed-point correctness, deterministic control, physical FPGA implementation, and transparent trade-offs. The next chapter establishes the technical and literature foundations for that design.

CHAPTER 2

BACKGROUND AND LITERATURE REVIEW

2.1 Introduction

This chapter develops the technical background for key-resource-aware candidate evaluation and positions **QFlow** within QKD networking, adaptive control, and reconfigurable computing. It also defines a comparison taxonomy that separates same-function hardware evidence from network simulations and component-level FPGA studies.

2.2 Trusted-Relay QKD Networks

QKD establishes secret material across a quantum channel and an authenticated classical channel. Networking adds key management, trusted relay operation, application interfaces, routing, monitoring, and recovery. Surveys describe QKD networks as layered systems in which physical links generate secret material and classical control software manages its use [1, 2]. In a trusted-relay architecture, intermediate sites are trusted with the relevant key-handling operations. The trust assumption differs from entanglement-repeater architectures and must be part of the security policy.

Once link post-processing completes, the KMS stores classical key data and metadata. For link (i, j) , useful control quantities include available key units $K_{ij}(t)$, pool capacity $K_{ij}^{\max}$, effective generation rate $\lambda_{ij}(t)$, consumption rate $C_{ij}(t)$, operational status, utilization, and health indicators derived from QBER or other monitored values. The physical fibre model may use attenuation coefficient α_{fiber} , but this quantity is distinct from controller scoring coefficients.

2.3 Key Pools and Resource Evolution

A discrete-time key-pool balance can be written as

$$K_{ij}[n+1] = \text{clip}_{[0, K_{ij}^{\max}]} (K_{ij}[n] + G_{ij}[n] - C_{ij}[n] - X_{ij}[n]), \quad (2.1)$$

where $G_{ij}[n]$ is newly admitted secret material, $C_{ij}[n]$ is consumed material, and $X_{ij}[n]$ is material removed through expiry, revocation, alarms, or administrative action. Every term must use the same unit, such as bits or fixed key blocks. A normalized occupancy is

$$o_{ij}[n] = \frac{K_{ij}[n]}{K_{ij}^{\max}}, \quad 0 \leq o_{ij}[n] \leq 1. \quad (2.2)$$

Generation and consumption rates convey information that occupancy alone cannot. Two links with equal current occupancy can have different future risk if one is replenishing quickly

and the other serves a high request load. A useful candidate descriptor therefore combines present reserve with bounded rate, load, and health information.

Freshness, cryptoperiod, or expiry is a KMS/security-policy property. A policy may invalidate material after a configured time, assign a soft preference to newer authorized batches, or revoke material after an alarm. These operations concern the management of classical keys; they are not quantum-state evolution in memory.

2.4 Hard Eligibility and Soft Preference

For a request requiring $D[n]$ key units, a directed link can be represented by the hard eligibility indicator

$$\chi_{ij}[n; D] = \mathbf{1}\{\text{status}_{ij} = \text{UP}\} \mathbf{1}\{K_{ij}[n] \geq D[n]\} \mathbf{1}\{q_{ij}[n] \leq q_{\text{abort}}\} \mathbf{1}\{\mathcal{M}_{ij}[n] \text{ is valid}\}, \quad (2.3)$$

where $\mathcal{M}_{ij}$ collects authenticated administrative metadata. Candidate $\mathcal{R}_i$ is feasible when every constituent link is eligible:

$$v_i[n] = \prod_{(u,v) \in \mathcal{R}_i} \chi_{uv}[n; D]. \quad (2.4)$$

Hard checks protect invariants. Soft preference is applied only after feasibility is known and expresses engineering priorities among admissible candidates. This separation avoids using a weighted score as a substitute for security or resource constraints.

2.5 Candidate-Route Evaluation

Candidate-generation algorithms may use shortest paths, constrained path search, multipath methods, optimization, or policy filters. **QFlow** receives the resulting bounded set $\mathcal{R}_{sd}$ rather than traversing the complete graph. Each candidate descriptor contains fixed-width fields such as normalized distance, bottleneck occupancy, replenishment condition, load or imbalance, operational risk, hop count, and identifier. Candidate evaluation then becomes a regular reduction: compute a score for every feasible descriptor and retain the best candidate under a deterministic tie-break rule.

The distinction between generation and evaluation is important for both science and implementation. Network-level papers may assess blocking, topology robustness, or global path quality, while an evaluator implementation may assess arithmetic exactness, cycles, post-route timing, and resources. The measurements are related but not interchangeable.

2.6 Multi-Objective Scoring

Key-aware selection combines objectives that have different units. Each metric is therefore normalized to a bounded dimensionless penalty before weighting. For candidate $\mathcal{R}_i$, a four-

term vector can be written as

$$\phi_i[n] = \begin{bmatrix} 1 - \hat{o}_i[n] \\ 1 - \hat{r}_i[n] \\ \hat{u}_i[n] \\ \hat{\rho}_i[n] \end{bmatrix}, \quad (2.5)$$

representing scarcity, recharge deficit, utilization or imbalance, and operational risk. Scoring profile Π_k supplies coefficient vector $\alpha^{(k)}$, giving local cost

$$c_i^{(k)}[n] = \text{sat}_{W_c} \left(\sum_{r=1}^4 \alpha_r^{(k)} \phi_{i,r}[n] \right). \quad (2.6)$$

Tchebycheff decomposition provides a scalar comparison while retaining an explicit multi-objective interpretation [14]. If $\hat{\mathbf{f}}_i^{(k)}$ is the normalized objective vector and $\mathbf{z}^*$ is an ideal point, the score is

$$g_i^{(k)}[n] = \max_j \left\{ \lambda_{\text{TCH},j}^{(k)} \left| \hat{f}_{i,j}^{(k)}[n] - z_j^* \right| \right\}. \quad (2.7)$$

The vector $\lambda_{\text{TCH}}^{(k)}$ contains objective weights; it is distinct from link key-generation rate $\lambda_{ij}(t)$. The use of Pareto and decomposition concepts is grounded in established multi-objective optimization methods [15, 14].

2.7 Offline Reinforcement Learning for Profile Selection

Reinforcement learning (RL) maps observed state to actions that maximize expected long-term reward. In tabular Q-learning, an offline training update is

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \eta_{\text{RL}} \left[r_t + \gamma_{\text{RL}} \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right], \quad (2.8)$$

where η_{RL} is the learning rate and γ_{RL} is the discount factor [16, 17]. In this thesis, the action does not directly construct a route. It selects one of four fixed scoring profiles. This reduces the deployed decision to a finite mapping from a quantized state to a two-bit action.

Training and inference are separated. Training explores traces, updates Q-values, and freezes a selected policy. Deployment stores only the policy mapping needed for deterministic inference. The FPGA therefore contains no exploration mechanism and no runtime Q-value update path. A minimum-dwell mechanism can suppress rapid profile changes by holding the current action until a specified number of accepted decisions has elapsed.

2.8 Fixed-Point Arithmetic and FPGA Implementation

Fixed-point arithmetic assigns an explicit integer and fractional width to every quantity. For a signed $Qm.n$ value, representable values and rounding rules depend on total width, signedness,

scaling, saturation, and conversion policy. Bit-exact verification requires these details to be identical in the software reference and RTL. Saturation is preferable to silent wraparound for bounded metrics because it preserves the intended ordering at numeric limits.

FPGA logic can implement quantizers, ROMs, adders, comparators, and finite-state control with a deterministic clocked interface. Look-up tables (LUTs), registers, occupied slices, block memories, and timing reports characterize the physical mapping. Post-synthesis timing is an estimate before placement and routing; post-route timing includes the implemented netlist and routing delays and is the appropriate source for the maximum-frequency values used in the final comparison.

2.9 CPU, FPGA, and ASIC Characteristics

A CPU offers mature software tools, flexible control flow, and efficient handling of irregular workloads. It is appropriate for topology management, candidate generation, KMS/SDN orchestration, offline training, logging, and security administration. Its execution time can depend on operating-system scheduling, caches, and memory behavior unless a real-time environment is used.

An FPGA supports reconfigurable spatial datapaths and deterministic state machines. It is attractive for an evolving fixed-point kernel because profiles and interfaces can be revised by loading a new bitstream. A dedicated ASIC may ultimately provide greater density, performance, and energy efficiency, especially at high volume, but requires larger non-recurring engineering effort, signoff, fabrication, packaging, test, and redesign if the contract changes. Platform comparison must therefore be specific to development stage, volume, and measurement boundary.

2.10 Literature Review

2.10.1 QKD routing and network control

Bi et al. use virtual-network state and key resources for dynamic QKD routing [5]. Chen et al. study hybrid-trusted path selection [18]; Kiktenko et al. examine multiple non-overlapping paths [6]; and Akhtar et al. present fast and secure routing algorithms for QKD networks [19]. Zhang et al. jointly optimize routing, channels, key rate, and time slots [7]. These studies show why hop count alone is insufficient, but their evaluated functions and evidence boundaries differ from a fixed-point candidate-evaluation kernel.

RL-based work includes deep-RL resource assignment [20], tabular Q-learning for resource-aware trusted-relay routing [8], and a graph-attention agent for flexible meshed topologies [9]. These studies provide broader network or generalization evidence than the present physical replay. They do not provide a same-input, same-function FPGA timing baseline.

2.10.2 Operational QKD systems

MadQCI demonstrates heterogeneous SDN-QKD integration in production facilities [3]. Chen et al. report a carrier-grade quantum communication network extending beyond 10,000 km [4]. Such systems establish the practical setting for KMS and controller integration. Their scale and end-to-end objectives do not make their performance numbers directly comparable with a bounded FPGA kernel.

2.10.3 FPGA acceleration in QKD

Reconfigurable hardware has accelerated multidimensional reconciliation encoding, QKD post-processing, polar-code information reconciliation, error correction, and synchronization [10, 11, 12, 21, 13]. These contributions demonstrate that FPGA implementation is practical in QKD systems, but they process different functions and use throughput or latency boundaries that cannot be converted into route-selection speedups.

Hardware genetic-algorithm research provides relevant implementation ideas for pipelined evaluation, selection, and specialization [22, 23, 24, 25, 26]. It remains component-level context because those systems do not evaluate trusted-relay key-resource descriptors.

2.10.4 Comparison taxonomy

Table 2.1. Comparison classes used throughout the thesis.

Class	Definition	Valid comparison
A	Same function, board, interface, numeric contract, replay, and measurement boundary	Direct timing, latency, resource, exactness, and controller-behavior comparison
B	Same function and inputs on CPU/software and FPGA	Direct speed or energy comparison only when both are measured under a matched protocol
C	QKD routing and network-control algorithms	Blocking, utilization, quality, robustness, or generalization under compatible network experiments
D	Different QKD components, FPGA/RL hardware, deployments, or VLSI kernels	Architectural and feasibility context; no raw QFlow speed ratio

Class A is supplied internally by the five same-board configurations. No Class B experiment was completed. The QKD routing studies are Class C, while FPGA post-processing, operational deployments, hardware genetic algorithms, and isolated VLSI kernels are Class D. This taxonomy prevents a latency ratio between systems that perform different computations.

2.10.5 Reproducible literature search and verification

The literature record was verified on 17 July 2026 from a predefined 50-row review matrix. Each row was searched by exact title at an official publisher database or the official arXiv record, including IEEE Xplore, ACM Digital Library, SpringerLink, Nature, MDPI, Wiley/IET, Optica, Elsevier ScienceDirect, INFORMS, and Google Books where appropriate. The audit checked bibliographic identity, platform, evidence layer, metric scope, and whether decision latency was actually reported. One audit identifier was retained per matrix row; sources whose exact identity could not be verified were excluded, and explicit replacement sources were recorded as replacements rather than silently treated as the same publication.

The resulting disposition contained 21 Class C and 29 Class D records, with no identified Class A or Class B publication in the documented search. This is a bounded finding from the recorded matrix and search date. Search queries, publisher locations, verification outcomes, and the final workbook are preserved under `literature/p1/` on the evidence branch identified in Appendix A.

2.10.6 Research position

Table 2.2. Representative related-work positioning. Direct numerical comparability requires an equivalent function and measurement boundary.

Work/category	Function	Evidence	Relationship to QFlow
Hao et al. [8]	Resource-aware adaptive routing	Network simulation	Class C RL counterpart; broader routing scope, no same-function FPGA
Johann et al. [9]	Generalizing DRL routing	Network simulation	Class C topology/generalization evidence; no direct kernel ratio
Bi et al. [5]	Dynamic key-resource routing	Software and network model	Class C resource-aware motivation
MadQCI [3]	SDN-QKD deployment	Operational testbed	Class D deployment context
Venkatachalam et al. [11]	QKD post-processing	FPGA implementation	Class D proof of FPGA suitability for a different function
This work	Fixed-point candidate evaluation	RTL, post-route, and physical replay	Class A internal same-board comparison with exact reference agreement

The research position is consequently specific. **QFlow** contributes physically validated fixed-point candidate-evaluation evidence, while several software studies contribute broader topology, algorithmic, or generalization evidence. Neither type of evidence dominates the other in every

dimension.

2.11 Data-Analysis Principles

The evaluation uses exact integer comparison for reference/RTL/board agreement; post-route reports for maximum frequency and resources; accepted-request-to-done cycles for kernel latency; profile counts, transitions, and dwell lengths for adaptive behavior; and paired confidence intervals, sign tests, and effect sizes for held-out route-quality comparisons. Missing power, energy, or CPU results are treated as unmeasured rather than as zero.

2.12 Conclusion

The background motivates key-resource-aware routing, establishes the distinction between hard eligibility and soft profile-conditioned preference, and explains how offline tabular Q-learning becomes deterministic policy-ROM inference. The literature taxonomy positions **QFlow** as a physically validated candidate evaluator without constructing invalid speed comparisons to network algorithms or different QKD components.

CHAPTER 3

METHODOLOGY

3.1 Introduction

This chapter defines the system boundary, classical key-resource model, candidate representation, scoring profiles, adaptive controller, hardware configurations, fixed-point implementation, verification, physical replay, statistical analysis, and supporting VLSI method used in the thesis.

3.2 System Overview

QFlow is placed between control-plane candidate generation and final policy validation. The KMS/SDN controller obtains authenticated network metadata and generates admissible candidate routes. The accelerator encodes the current state, selects one scoring profile, evaluates the supplied candidates, and returns a candidate identifier, score, profile identifier, and status. The controller remains responsible for security checks, fallback, and route installation.

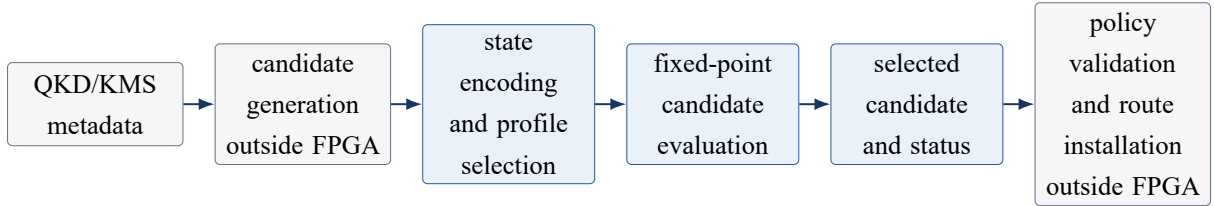


Figure 3.1. End-to-end methodology and the FPGA kernel boundary.

3.3 System Boundary

The accelerator receives fixed-width descriptors and bounded controller features. It does not receive secret-key bits, discover network topology, generate arbitrary paths, operate the KMS, update an RL Q-table, or install a route. Its output is a recommendation within a deterministic arithmetic contract. This boundary is applied consistently to software, RTL, post-route, and physical measurements.

3.4 Network and Key-Resource Model

The trusted-relay network is a directed graph $G = (V, E)$. At decision epoch n , link (i, j) is represented by

$$\xi_{ij}[n] = (K_{ij}[n], K_{ij}^{\max}, \lambda_{ij}[n], C_{ij}[n], q_{ij}[n], \text{status}_{ij}[n], u_{ij}[n], \mathcal{M}_{ij}[n]), \quad (3.1)$$

where K_{ij} is available classical key material, λ_{ij} is effective key-generation rate, C_{ij} is consumption, q_{ij} is a QBER-derived or other external health indicator, u_{ij} is utilization, and $\mathcal{M}_{ij}$ contains authenticated policy metadata. The pool follows the balance in (2.1).

For a demand $D[n]$, hard eligibility is

$$\chi_{ij}[n; D] = \mathbf{1}\{\text{status}_{ij} = \text{UP}\} \mathbf{1}\{K_{ij}[n] \geq D[n]\} \mathbf{1}\{q_{ij}[n] \leq q_{\text{abort}}\} \mathbf{1}\{\mathcal{M}_{ij}[n] \text{ is valid}\}. \quad (3.2)$$

Policy metadata may encode authorization, provenance, expiry, revocation, or an alarm. Optional freshness is a KMS/security rule. It is not modelled as quantum decoherence of stored classical keys.

3.5 Candidate-Route Representation

For each request, software supplies

$$\mathcal{R}_{sd} = \{\mathcal{R}_0, \mathcal{R}_1, \dots, \mathcal{R}_{m-1}\}. \quad (3.3)$$

Candidate $\mathcal{R}_i$ is described by

$$\mathbf{c}_i = \left(v_i, \hat{d}_i, \hat{o}_i, \hat{r}_i, \hat{u}_i, \hat{\rho}_i, h_i, \text{id}_i \right), \quad (3.4)$$

where v_i is feasibility; $\hat{d}_i$ is normalized distance or consumption cost; $\hat{o}_i$ is bottleneck occupancy; $\hat{r}_i$ is replenishment or effective key-generation condition; $\hat{u}_i$ is utilization or imbalance; $\hat{\rho}_i$ is bounded operational risk; h_i is hop count; and id_i is the deterministic tie-break identifier. Hatted fields are dimensionless and saturated to their declared ranges.

3.6 Eligibility Rules

Candidate feasibility is computed before scoring:

$$v_i[n] = \prod_{(u,v) \in \mathcal{R}_i} \chi_{uv}[n; D]. \quad (3.5)$$

An infeasible candidate cannot win through soft weights. If no candidate is feasible, the kernel returns a no-path status. Invalid encodings return an invalid status. Exact status behavior and request timing are specified in Appendix B.

3.7 Fixed-Point Candidate-Scoring Model

For candidate $\mathcal{R}_i$, the normalized soft-penalty vector is

$$\phi_i[n] = \begin{bmatrix} 1 - \widehat{o}_i[n] \\ 1 - \widehat{r}_i[n] \\ \widehat{u}_i[n] \\ \widehat{\rho}_i[n] \end{bmatrix}. \quad (3.6)$$

For active profile Π_k , the saturated local cost is

$$c_i^{(k)}[n] = \text{sat}_{W_c} \left(\sum_{r=1}^4 \alpha_r^{(k)} \phi_{i,r}[n] \right). \quad (3.7)$$

The first path objective is the normalized local cost; the remaining objectives represent bottleneck reserve and utilization balance:

$$\widehat{\mathbf{f}}_i^{(k)}[n] = \begin{bmatrix} \mathcal{N}_c(c_i^{(k)}[n]) \\ 1 - o_{\min}(\mathcal{R}_i, n) \\ \widehat{I}(\mathcal{R}_i, n) \end{bmatrix}. \quad (3.8)$$

The Tchebycheff score is

$$g_i^{(k)}[n] = \max_{j \in \{1,2,3\}} \left\{ \lambda_{\text{TCH},j}^{(k)} \left| \widehat{f}_{i,j}^{(k)}[n] - z_j^* \right| \right\}. \quad (3.9)$$

Among feasible candidates, the minimum score wins; an exact tie is resolved by the smallest candidate identifier. Scores are compared only within the active profile and fixed numeric contract.

3.8 Fixed-Point Scoring-Profile Codebook

The four profiles provide distinct scoring emphases. Their coefficient values are encoded in half-unit steps, and their Tchebycheff ratios use two-bit unsigned integers. Table 3.1 defines each profile once for the remainder of the thesis.

Table 3.1. Fixed-point scoring-profile codebook.

Profile	Intended emphasis	$\alpha^{(k)}$	$\lambda_{\text{TCH}}^{(k)}$	HW ratio
Π_0	Balanced operation	(1.0, 1.5, 0.5, 2.0)	(0.40, 0.40, 0.20)	(2, 2, 1)
Π_1	Key-scarcity protection	(1.5, 1.5, 1.0, 2.0)	(0.25, 0.25, 0.50)	(1, 1, 2)
Π_2	Operational-health protection	(1.0, 2.0, 0.5, 3.0)	(0.25, 0.50, 0.25)	(1, 2, 1)
Π_3	Low evaluation cost/latency emphasis	(0.5, 1.0, 0.5, 1.0)	(0.50, 0.25, 0.25)	(2, 1, 1)

The profile payload packs four three-bit α numerators followed by three two-bit Tchebycheff ratios:

$$\underbrace{\alpha_1[2:0] \alpha_2[2:0] \alpha_3[2:0] \alpha_4[2:0]}_{12 \text{ bits}} \underbrace{\lambda_1[1:0] \lambda_2[1:0] \lambda_3[1:0]}_{6 \text{ bits}}. \quad (3.10)$$

The four 18-bit words are 13329, 1B516, 14399, and 0A2A5 in hexadecimal.

3.9 Hardware Evaluation Configurations H0–H4

The letter H denotes a hardware evaluation configuration. Table 3.2 defines the hardware evaluation configurations H0–H4 and their roles. Each configuration was synthesized, placed, routed, and programmed independently.

Table 3.2. Hardware evaluation configurations H0–H4.

Config.	Selection method	Profile	Learning on FPGA	Additional hardware feature	Purpose
H0	Feasible shortest distance	None; minimal comparison	No	Feasibility and distance comparator	Minimal timing/resource reference
H1	Fixed key-aware baseline	Fixed	No	Key-aware weighted selection	Reference for adding resource awareness
H2	Fixed-profile QFlow evaluator	Fixed Π_0	No	Complete point datapath	Reference for profile-based scoring
H3	Threshold/rule-adaptive QFlow evaluator	Adaptive	No	Threshold encoder and rule controller	Non-learning adaptive-controller baseline
H4	Reinforcement-learned adaptive QFlow evaluator	Adaptive	No; deterministic inference only	Offline-trained policy ROM and minimum-dwell control	Learned controller comparison against configuration H3

Configuration H0 is deliberately minimal and is not feature-equivalent to configurations H1–H4. Consequently, the most meaningful adaptive-controller comparison is configuration H4 versus configuration H3. Configuration H4 reads a fixed policy ROM; the FPGA neither stores training traces nor updates Q-values.

All configurations share the external shell, candidate widths, result schema, 100 MHz operating constraint, replay rows, and accepted-request-to-done measurement boundary.

3.10 Controller State Representation

The adaptive observation is

$$\mathbf{x}[n] = (x_B[n], x_R[n], x_L[n], x_I[n]), \quad (3.11)$$

where x_B is minimum normalized candidate key-pool occupancy, x_R is normalized replenishment/effective key-generation condition, x_L is offered request load, and x_I is utilization imbalance. Each feature is clipped and assigned to one of four bins using three fixed thresholds.

Equality enters the higher bin.

The bins are packed in radix four:

$$z[n] = ((4b_B + b_R)4 + b_L)4 + b_I, \quad z[n] \in \{0, \dots, 255\}. \quad (3.12)$$

Four bins for each of four features produce $4^4 = 256$ states. The action $a[n] \in \{0, 1, 2, 3\}$ selects profile $\Pi_{a[n]}$.

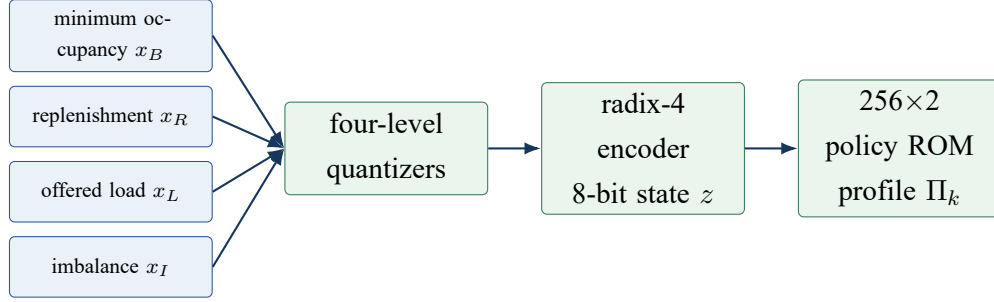


Figure 3.2. Four-feature state encoding and policy-ROM inference in configuration H4.

3.11 Offline Policy Learning

Eight tabular Q-learning candidates used trainer seeds 17, 29, 43, 71, 113, 167, 229, and 283. Each candidate trained for 200 epochs and 819,200 transitions. The learning rate was $\eta_{\text{RL}} = 0.1$, the discount factor was $\gamma_{\text{RL}} = 0.95$, and exploration decreased linearly from 1.0 to 0.05 over the first 80% of training transitions.

Training, validation, and test traces were separated. Candidate eligibility required at least two used profiles, at least 1% use of the second-most-used profile, blocking within an absolute 0.005 margin, and validation switch rate no greater than 0.25. Eligible candidates were ranked by validation reward, blocking, quality outcome, switching, and trainer seed. Seed 229 was selected and its policy, semantic description, and file hashes were fixed before held-out evaluation.

Pseudocode 3.1. Offline training, selection, and policy freeze

Input: training traces $\mathcal{D}_{\text{tr}}$, validation traces $\mathcal{D}_{\text{val}}$, and trainer seeds $\mathcal{S}$.

Output: frozen policy π_{229} .

1. Initialize one tabular Q-function for every seed in $\mathcal{S}$.
2. Apply Q-learning updates over the training traces for 200 epochs.
3. Evaluate each final policy on validation traces and record decomposed reward, blocking, profile use, and switching.
4. Apply the declared eligibility rules and rank the eligible policies.

5. Select seed 229; record configuration and file hashes.
6. Lock the policy before accessing the held-out traces.

3.12 Policy and Profile ROM Deployment

The deployed policy ROM contains 256 state-major two-bit actions. The profile ROM contains four 18-bit words. A 1,024-word signed Q8.7 table was retained for training analysis but is not read by the deployed controller. Replacing runtime argmax over four Q-values with the frozen two-bit action removes unnecessary storage and comparison logic while preserving the selected policy.

3.13 Minimum-Dwell Mechanism

Let $a^*[n] = \pi(z[n])$ be the requested policy action and $\delta[n]$ the number of accepted decisions since the last profile change. The deployed action is

$$a[n] = \begin{cases} a[n-1], & a^*[n] \neq a[n-1] \wedge \delta[n] < d_{\min}, \\ a^*[n], & \text{otherwise,} \end{cases} \quad (3.13)$$

with $d_{\min} = 3$. Dwell is counted in accepted decisions, not idle clock cycles. The mechanism is evaluated as part of the complete configuration H4 controller; the experiment does not isolate policy and dwell contributions.

3.14 RTL Architecture

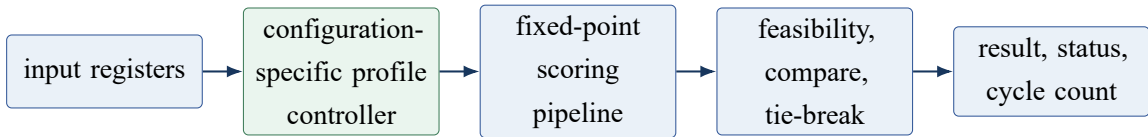


Figure 3.3. Common candidate-evaluation RTL architecture.

A request is accepted when `start` is asserted while the core is ready. Inputs are sampled on that edge and held internally. The `done` signal marks the synchronous completion edge with stable output fields. Reset, invalid input, no-path, back-to-back request, and start-while-busy behavior are part of the verification contract.

3.15 Fixed-Point Representation

The reference and RTL use explicit width, signedness, scaling, rounding, saturation, packing order, and tie-breaking rules. The four-profile payload contains 72 useful bits in total, and the action uses two useful bits. State IDs and output fields are compared as exact integers rather than floating-point approximations. Appendix B records the storage and protocol layout.

3.16 Experimental Inputs, Workloads, and Trace Partitions

The study used controlled traces and candidate records rather than an observational dataset. Table 3.3 summarizes their roles.

Table 3.3. Experimental inputs and trace partitions.

Partition	Size	Purpose
Controller state space	256 states and 36 threshold-edge cases	Exhaustive state, threshold, ROM, and action verification
Offline training	Eight candidates, 200 epochs, 819,200 transitions each	Learn tabular policies without held-out access
Validation	Four traces, 512 decisions each	Policy eligibility and selection
Held-out software	Four traces, 512 decisions each	Compare fixed, rule-adaptive, and learned controllers after freeze
Physical contract replay	20 rows per configuration	Directed protocol and functional boundary cases
Physical held-out replay	64 rows per configuration	Common same-board comparison
Physical corpus	84 rows $\times$ five configurations	420 records; nine checked fields per record

3.17 Verification Methodology

Verification proceeded from the mathematical reference to the physical board:

1. fixed-point software reference checks;
2. exhaustive state encoder, policy ROM, and profile ROM tests;
3. cycle-accurate unit and integrated RTL simulation;

4. independent synthesis, mapping, placement, routing, and timing for every hardware configuration; and
5. physical UART replay with field-by-field comparison.

Controller verification comprised 1,509 cycle vectors and 302 valid decisions, including all 256 states, 36 threshold-edge cases, all four actions, dwell holds and switches, invalid input, no path, reset, pipeline wait, and busy behavior. Seventy-four Python regression tests passed. Integrated tracing added 2,125 cycle vectors and 530 valid decisions. Evidence type was recorded separately for software, RTL, post-synthesis, post-route, and physical-board results.

3.18 FPGA Implementation

The target was a Digilent Nexys 3 Rev B with a Xilinx XC6SLX16-2-CSG324 Spartan-6 FPGA. Xilinx ISE 14.7 independently built configurations H0–H4 with the same board shell, UART protocol, 100 MHz user clock, candidate widths, output schema, and replay rows. The 100 MHz value is the operating contract; maximum frequency was extracted from each post-route timing report.

3.19 Physical Replay Protocol

For each configuration, the host programmed the corresponding bitstream, transmitted one fixed record at a time, captured the raw UART response, parsed the record, and compared nine output fields with the reference. Eighty-four records produced 756 checked fields per configuration and 3,780 across the five configurations.

Pseudocode 3.2. Physical replay and exact comparison

Input: configuration h , replay rows $\mathcal{D}$, and expected results Y_h .

Output: raw capture, parsed records, and exactness summary.

1. Program the independently generated bitstream for configuration h .
2. Transmit each encoded replay row and retain the raw response.
3. Parse result, status, profile, score, identifier, and cycle fields.
4. Compare all nine checked fields with Y_h .
5. Record row count, exact fields, mismatches, and protocol status.
6. Hash the capture, parsed data, implementation reports, and summary.

Kernel latency for record i is

$$L_i = c_{\text{done},i} - c_{\text{accept},i}, \quad t_i = L_i T_{\text{clk}}. \quad (3.14)$$

The boundary excludes UART serialization, host parsing, operating-system scheduling, JTAG programming, KMS acquisition, candidate generation, security validation, and route installation.

3.20 Statistical Analysis

Exactness required equality of every checked integer field. Timing and resource metrics came directly from independent implementation reports. Adaptive behavior was summarized over valid decisions by profile counts, transitions, dwell runs, and dwell lengths.

For held-out route quality, only jointly successful pairs were compared. The analysis reported pair count, paired mean difference, bootstrap 95% confidence interval, two-sided sign-test p -value, paired effect size d_z , and path-change count. The limited number of distinct replay rows and repeated structure can reduce statistical power and independence. Scores formed under different active profiles were not treated as directly comparable physical units.

3.21 Supporting VLSI Methodology

The supporting VLSI study used the version-controlled evidence snapshot identified in Appendix A. FDPE-V0–V3, SKAG-W0–W1, and Pareto-C0–C1 were synthesized with Yosys, and generic cell counts were read from the version-controlled synthesis logs. Generic cells are used only for within-flow structural comparisons; they are not FPGA LUTs.

Representative kernels SKAG-W1, FDPE-V3, and Pareto-C0 were separately processed through a SKY130 high-density OpenROAD flow comprising synthesis, floorplanning, placement, clock-tree synthesis, routing, parasitic extraction, and final report generation. Their areas refer to isolated routed kernels, not an integrated **QFlow** ASIC. FDPE-V3 retains its repository identifier as a supplementary arithmetic interpolation experiment and is not presented as current candidate-evaluator functionality.

A representative 2:1 transmission-gate multiplexer was also simulated in ngspice with simplified Level-1 MOS models at 1.8 V and loads of 5, 10, 20, and 50 fF. The calculated $\frac{1}{2}CV^2$ quantity describes only the stated capacitive load, not accelerator energy per decision.

3.22 Methodological Limitations

The physical replay used one board and device family, 84 rows per configuration, and a bounded candidate interface. The learned policy was trained on a controlled research environment and was not evaluated across a large set of unseen production topologies. No matched CPU timing experiment or activity-calibrated FPGA power measurement was completed. Live QKD devices, KMS/SDN orchestration, authenticated host transport, and route installation were outside the experiment. The VLSI work is isolated-kernel physical-design evidence rather than full-chip signoff.

3.23 Conclusion

The methodology combines a classical key-resource model, explicit eligibility, profile-conditioned fixed-point scoring, four-feature state encoding, offline policy learning, deterministic ROM inference, and independent same-board implementation. Its verification and statistical procedures distinguish exact hardware correctness from network-level route-quality evidence.

CHAPTER 4

RESULTS AND DISCUSSION

4.1 Introduction

This chapter reports software, RTL, post-route, and physical-board results in their respective evidence categories. The central result is a same-board comparison of five independent fixed-point candidate-evaluation configurations.

4.2 Experimental Overview

The evaluation followed four linked stages. Offline software training and held-out traces established controller behavior. Exhaustive and integrated RTL tests checked state encoding, policy lookup, dwell control, fixed-point arithmetic, and protocol timing. Independent Xilinx ISE builds provided post-route timing and resource data. Finally, the same Nexys 3 board replayed a common physical corpus for all five configurations.

The strongest comparison is internal Class A evidence: identical board, shell, clock, widths, record schema, and measurement boundary. Route-quality statistics are reported separately from exactness and implementation metrics.

4.3 Offline Controller Behavior

Each of the eight policy candidates completed 819,200 training transitions. Seven passed the validation criteria, and seed 229 was selected by the declared ranking. Table 4.1 summarizes the four-trace held-out software partition after the policy was fixed. All controllers completed 2,048 successful decisions with zero blocking in this controlled environment.

Table 4.1. Software-simulated held-out controller behavior over four 512-decision traces.

Controller	Reward	Block	Quality	Balance	Hops	Switch rate
Fixed-profile	4.6130	0	0.95815	0.16881	2.8750	0
Rule-adaptive	4.6292	0	0.95943	0.25513	2.9775	0.2813
Offline-learned adaptive	4.6771	0	0.96044	0.26499	2.9541	0.2329

The learned policy had the highest descriptive mean reward and balance utility in this software partition. With only four paired traces, the exact two-sided permutation p -value was 0.125 and the Holm-adjusted value was 0.25 for both learned comparisons. The software result therefore describes behavior under the selected environment; it is not evidence of statistical superiority.

4.4 Controller and Policy Verification

Table 4.2. Controller and policy verification summary.

Verification item	Count/result	Outcome
Encoded state addresses	256/256	Exact policy/state mapping
Threshold below/equal/above cases	36	Equality entered the higher bin
Profile actions	4/4	All profiles covered
Cycle vectors	1,509	Zero RTL mismatch
Valid decisions	302	Zero unexplained mismatch
Python regression tests	74	Pass
Lint warnings	0	Pass
Controller-only synthesis estimate	103 LUTs, 79 registers	Synthesis pass

The controller-only synthesis reported an estimated maximum frequency of 247.812 MHz. This value is a post-synthesis estimate for the isolated controller and is not used as the system frequency. Final maximum-frequency results are taken from complete post-route implementations.

4.5 Integrated RTL Verification

Integrated cycle-accurate tracing added 2,125 cycle vectors and 530 valid decisions. The valid-request controller path had a deterministic three-cycle latency. Reset, busy, no-path, invalid-input, threshold equality, dwell hold, dwell switch, and pipeline-wait cases were exercised. Candidate-evaluator regression covered 519 of 519 vectors, and the integrated policy shell matched 101 of 101 expected results. These are RTL-simulated results; physical exactness is established by the board replay below.

4.6 Independent FPGA Implementation

Configurations H0–H4 were built independently for the XC6SLX16-2-CSG324 device. Each build passed synthesis, mapping, placement, routing, and the 100 MHz timing constraint. Independent bitstreams prevent a parameter-only simulation from being reported as multiple physical implementations. Configuration H0 remained the minimal feasibility/distance reference, while configurations H1–H4 added the key-aware and profile-scoring functions defined in Table 3.2.

4.7 Same-Board Physical Replay and Exactness

Each configuration produced 84 physical records: 20 directed contract records and 64 held-out records. Nine output fields were checked per record. Table 4.3 reports the exact comparison.

Table 4.3. Physical replay exactness for the five independent configurations.

Configuration	Records	Fields/record	Exact fields	Mismatches
H0	84	9	756	0
H1	84	9	756	0
H2	84	9	756	0
H3	84	9	756	0
H4	84	9	756	0
Total	420	—	3,780	0

This result establishes that physical transport and parsing functioned, each independent implementation matched its fixed-point contract, and configuration H4 performed policy/profile inference consistently with the reference. Exactness does not by itself establish that one controller selects higher-quality routes.

4.8 Post-Route Timing and Kernel Latency

Table 4.4. Post-route maximum frequency and measured kernel latency under the common 100 MHz operating contract.

Configuration	Selection method	$f_{\max}$ (MHz)	Mean cycles	Latency (μ s)
H0	Feasible shortest distance	118.948	2.000	0.020
H1	Fixed key-aware baseline	107.538	201.763	2.018
H2	Fixed-profile QFlow	102.838	202.763	2.028
H3	Threshold/rule adaptive	102.648	202.763	2.028
H4	Reinforcement-learned adaptive	102.020	204.763	2.048

All configurations exceeded 100 MHz. Configuration H4 achieved a post-route maximum frequency of 102.020 MHz and a mean normal-case latency of 2.047632 μ s, rounded to 2.048 μ s. This is the accepted-request-to-done latency of the tested FPGA kernel. UART serialization, host processing, KMS acquisition, candidate generation, security checks, and route installation are outside the measurement.

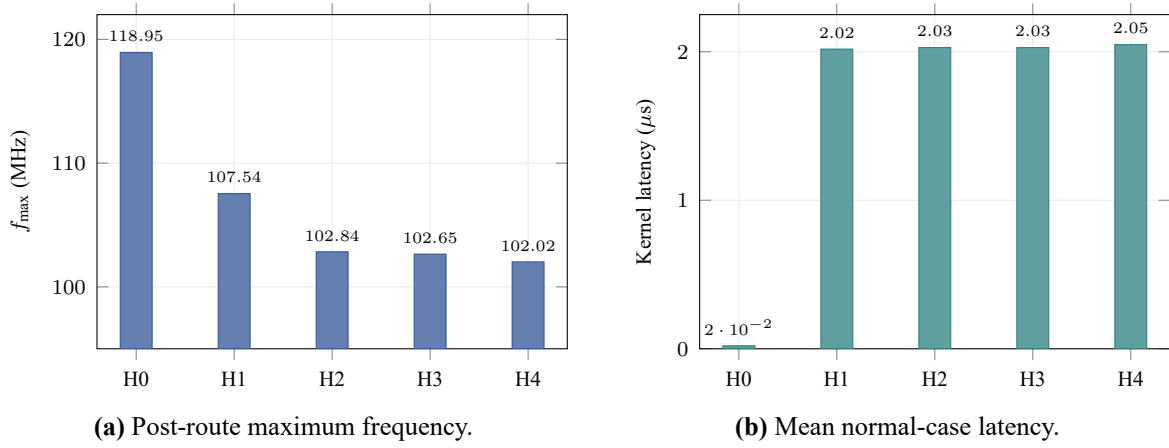


Figure 4.1. Timing and latency across the H0–H4 comparison.

The reciprocal of the rounded configuration H4 kernel latency is $1/(2.048 \mu s) \approx 488,281$ transactions/s. This is a theoretical sequential kernel-service rate under immediate back-to-back acceptance, not measured end-to-end throughput.

Configuration H0 is not feature-equivalent to the remaining configurations; its two-cycle result should not be used to calculate an RL overhead. The relevant adaptive comparison is configuration H4 versus configuration H3.

4.9 Resource Utilization

Table 4.5. Post-route resource utilization of independent H0–H4 implementations.

Configuration	Slice registers	Slice LUTs	Occupied slices
H0	493	801	301
H1	1,000	1,484	589
H2	1,085	1,650	649
H3	1,097	1,836	645
H4	1,156	1,706	677

Configuration H4 used 1,706 LUTs, 1,156 registers, and 677 occupied slices. Relative to configuration H3, it mapped to 7.081% fewer LUTs, 5.378% more registers, and 4.961% more occupied slices. LUT count and occupied slices can move in opposite directions because slice packing, register placement, and routing affect physical occupancy. The LUT reduction is an implementation-specific synthesis result, not a general property of reinforcement learning.

4.10 Adaptive Profile Behavior

Table 4.6. Profile use and switching over 81 valid adaptive decisions.

Configuration	Profile count				Transitions	Runs	Min dwell	Max dwell
	Π_0	Π_1	Π_2	Π_3				
H3	24	36	18	3	44	45	1	7
H4	42	11	18	10	24	25	1	7

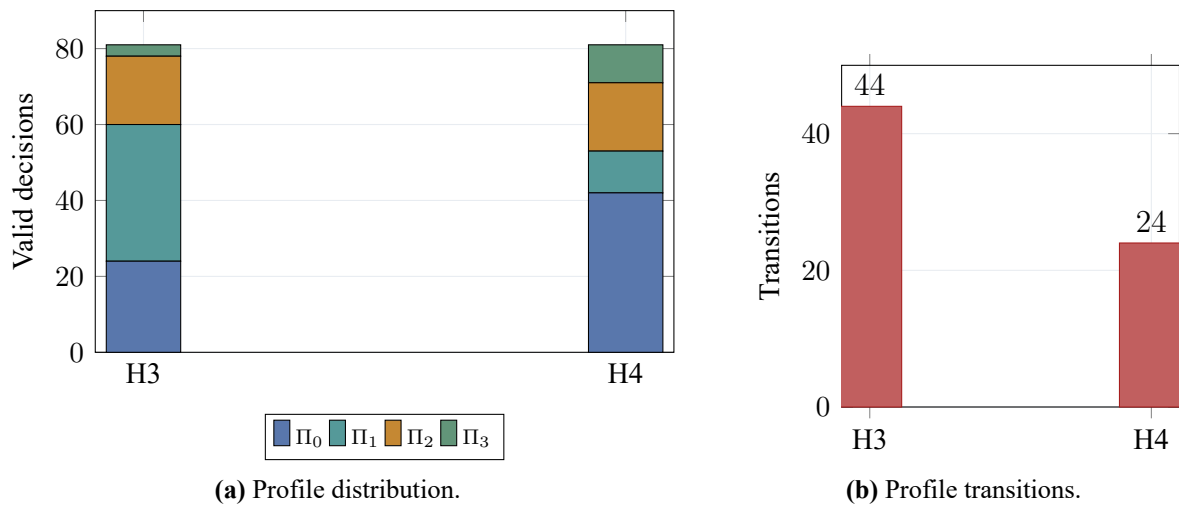


Figure 4.2. Trace-specific adaptive behavior of configurations H3 and H4.

The complete configuration H4 controller, including the offline-learned policy ROM and minimum-dwell mechanism, reduced observed profile transitions from 44 to 24. This is a 45.455% reduction on the evaluated replay, while mean dwell increased from 1.80 to 3.24 valid decisions. Isolating the learned policy contribution would require a policy-by-dwell ablation.

4.11 Held-Out Route-Quality Analysis

All configurations had 58 successful, four no-path, and two invalid rows in the 64-row held-out physical partition. Route choices could still differ within jointly successful pairs. Table 4.7 reports comparisons of configuration H4 with configurations H2 and H3.

Table 4.7. Paired held-out physical route-quality comparisons.

Comparison	Pairs	Mean diff.	95% CI	p	d_z	Path changes
H4-H2	58	86.47	$[-23.52, 228.22]$	0.2188	0.176	6
H4-H3	58	-78.05	$[-242.53, 79.93]$	0.4531	-0.127	7

Both confidence intervals cross zero. The p -values are 0.2188 for configuration H4 versus configuration H2 and 0.4531 for configuration H4 versus configuration H3. The limited number of rows and possible dependence among replay rows reduce statistical power. Configuration H4 demonstrated physical deployability and distinct adaptive behavior, but the available held-out physical comparisons do not establish statistically significant route-quality superiority over configurations H2 or H3.

4.12 H4 Versus H3 Engineering Trade-Off

Table 4.8. Configuration H4 relative to rule-adaptive configuration H3.

Metric	Change	Interpretation
Mean normal-case latency	+0.986%	Small additional policy/control latency
Post-route $f_{\max}$	−0.612%	Both configurations remain above 100 MHz
Slice LUTs	−7.081%	Implementation-specific mapping result
Slice registers	+5.378%	Additional registered control/state
Occupied slices	+4.961%	Increased physical footprint after packing/routing
Profile transitions	−45.455%	Complete policy-and-dwell controller switched less
Mean route-quality code	−78.05 units	Small, non-significant paired difference

The engineering contribution of configuration H4 is a bit-exact adaptive control law with microsecond-scale latency and explicit physical costs. Its value does not depend on winning every route-quality comparison.

4.13 Comparison with Related Work

Table 4.9 compares evidence dimensions without constructing speedup ratios across different functions.

Table 4.9. Function- and evidence-aware comparison with representative related work.

Work/category	Evaluated function	Platform	Evidence type	Topology/ evaluation scale	HW impl.	Board	Exact fixed point	Directly comparable
Bi et al. [5]	Dynamic key-resource routing	Software	Network model	Simulated QKD topology	No	No	No	No
Hao et al. [8]	Resource-aware Q-learning routing	Software	Network simulation	Trusted-relay simulation	No	No	No	No
Johann et al. [9]	Generalizing routing	DRL CPU/software	Network simulation	Multiple meshed topologies	No	No	No	No
MadQCI [3]	SDN-QKD integration	integrated system	Distributed system	Operational deployment	Multi-site production facilities	No	No	No
Venkatachalam et al. [11]	QKD post-processing	FPGA	Component implementation	Post-processing workload	Yes	Yes	Different contract	No
QFlow (this work)	Route-candidate evaluation	Spartan-6 FPGA	Post-route and physical replay	420 records across five builds	Yes	Yes	Yes	Internal H0–H4 only

QFlow is stronger in physically validated fixed-point candidate-evaluation evidence, while several software studies provide broader topology, network-level, or generalization evidence. Direct numerical comparison is limited to the internal same-board configurations.

4.14 Practical Deployment Scenario

A realistic deployment would partition responsibilities as follows:

1. The KMS/SDN controller receives key-pool occupancy, effective key-generation rate, link activity, QBER-derived or other externally measured health indicators, request load, and utilization.
2. The controller generates admissible route candidates from the current topology and policy.
3. Candidate descriptors and state metadata are sent to **QFlow**.
4. **QFlow** encodes the state, selects profile Π_k , evaluates the candidates, and returns a candidate identifier, profile, score, and status.
5. The controller checks administrative and security policy, installs the selected route, and handles failure or fallback.

The $2.048 \mu s$ value applies only to step 4 within the tested FPGA boundary. It is not the time to collect KMS data or install a network route. The practical value of the accelerator lies in deterministic recurring computation, fixed-point reproducibility, predictable kernel latency, and reconfigurable profile deployment.

4.15 Platform Selection and Cost Implications

4.15.1 CPU role

CPU/software remains preferable for topology management, KMS and SDN orchestration, candidate generation, policy management, offline Q-learning, logging, security administration, and irregular or infrequently executed tasks. No same-computation, same-input CPU timing experiment was completed. Accordingly, this thesis reports no CPU-to-FPGA speedup and no CPU/FPGA energy comparison.

4.15.2 FPGA role

The measured configuration H4 evidence comprises 102.020 MHz post-route maximum frequency, 2.048 μ s mean kernel latency, 1,706 LUTs, 1,156 registers, 677 occupied slices, and exact hardware/reference agreement. FPGA implementation was appropriate for the present research stage because the architecture and profiles remain evolvable, bitstreams can be changed after deployment, configurations H0–H4 can be implemented on the same device, deterministic pipelines are available, and physical implementation evidence can be collected before committing to silicon.

4.15.3 ASIC role

A mature ASIC could potentially provide lower energy per transaction, higher density, greater performance, and lower high-volume unit cost. It would require substantial non-recurring engineering, complete signoff, fabrication, packaging, testing, longer development time, stronger verification, redesign cost when the model changes, and loss of post-fabrication reconfigurability.

For the present low-volume, evolving research prototype, FPGA is the more appropriate engineering platform. For a stable, high-volume commercial product, ASIC implementation may eventually be more economical and energy-efficient.

4.15.4 Conceptual total-cost model

Let production volume be N . A simple platform cost model is

$$C_{\text{FPGA}}(N) = C_{\text{dev,FPGA}} + N c_{\text{FPGA}}, \quad (4.1)$$

$$C_{\text{ASIC}}(N) = C_{\text{NRE,ASIC}} + N c_{\text{ASIC}}, \quad (4.2)$$

where $C_{\text{dev,FPGA}}$ is FPGA development and integration cost, $C_{\text{NRE,ASIC}}$ includes ASIC design, verification, masks, fabrication, packaging, and test, and c_{FPGA} and c_{ASIC} are per-unit costs.

When $c_{\text{FPGA}} > c_{\text{ASIC}}$, the conceptual break-even volume is

$$N^* = \frac{C_{\text{NRE,ASIC}} - C_{\text{dev,FPGA}}}{c_{\text{FPGA}} - c_{\text{ASIC}}}. \quad (4.3)$$

Below N^* , FPGA can have lower total cost; above it, a stable ASIC may become economical. The thesis does not estimate N^* because verified commercial cost inputs are unavailable.

Table 4.10. Qualitative platform comparison for the **QFlow** development stages.

Criterion	CPU	FPGA	ASIC
Development effort	Low to moderate	Moderate	High
Reconfigurability	Software update	Bitstream update	None after fabrication
Deterministic execution	Requires careful system design	Natural for fixed pipeline	Natural for fixed design
Low-volume total cost	Usually favourable	Favourable for hardware prototype	Usually unfavourable
High-volume unit cost	Processor/platform dependent	Often higher	Potentially lowest
Energy potential	General-purpose overhead	Specialized datapath potential	Highest specialization potential
Time to deployment	Short	Moderate	Long
Current QFlow suitability	Control plane and training	Candidate-evaluation prototype	Premature
Future mature QFlow	Orchestration and fallback	Reconfigurable product option	Potential high-volume option

4.16 Kernel-Level Physical-Design Exploration

The supporting VLSI study examined selected computation kernels rather than a complete accelerator. Yosys generic synthesis reduced the SKAG scoring kernel from 5,343 cells with runtime products to 503 cells with fixed-coefficient shift-add logic, a 90.59% reduction within that flow. This result illustrates the structural benefit of coefficient specialization; generic cells are not FPGA LUTs.

Table 4.11. OpenROAD physical-design results for isolated kernels.

Kernel	Evaluated role	Area (μm^2)	Utilization
SKAG-W1	Fixed-coefficient shift-add scoring	6,451	26%
FDPE-V3	Supplementary LUT/interpolation arithmetic	22,186	13%
Pareto-C0	Dominance and selection comparator	3,368	13%

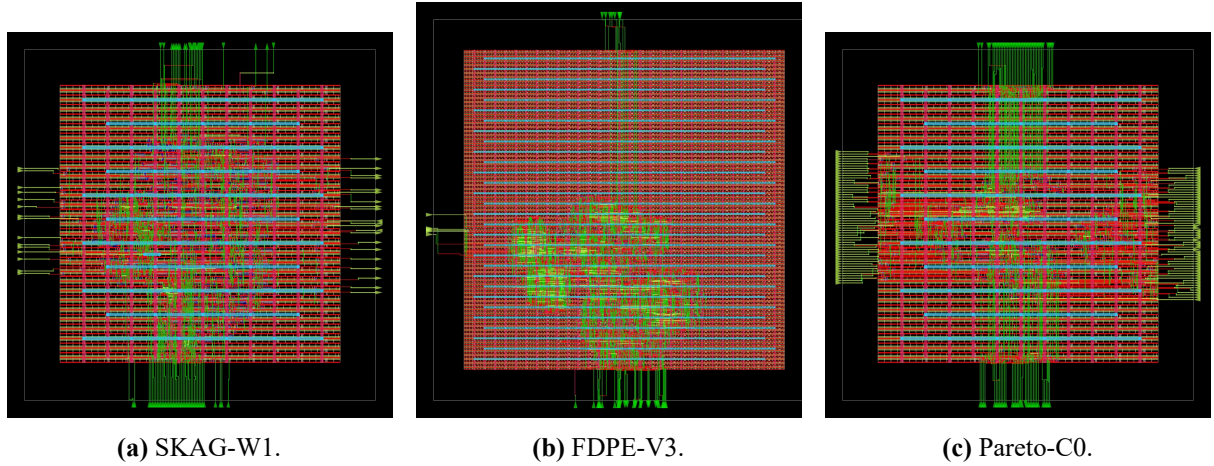


Figure 4.3. Routed OpenROAD layouts for three isolated kernels. These views do not represent an integrated **QFlow** ASIC.

The flows generated routed reports and final artifacts for the separate kernels, showing that selected functions can pass through an open-source physical-design workflow. The areas must not be summed as a **QFlow** chip area. The study does not provide full-chip power, commercial DRC/LVS signoff, tapeout, or fabricated-silicon performance.

4.17 Threats to Validity

- The physical experiment used one FPGA board and device family.
- The physical corpus contained 84 rows per configuration and a bounded candidate set.
- The offline-learned policy was evaluated in a controlled research environment rather than across many independent unseen topologies.
- Replay rows may share structure, reducing the effective independence and power of route-quality tests.
- No matched CPU benchmark, activity-calibrated power measurement, or live KMS/SDN deployment was available.
- Configuration H4 performed inference only; online learning was not tested.

- The OpenROAD results concern isolated kernels and do not establish a complete ASIC implementation.

4.18 Answers to the Research Questions

Table 4.12. Answers to the research questions.

RQ	Answer	Evidence
RQ1	All five independent configurations exceeded the 100 MHz post-route target.	Table 4.4
RQ2	All 420 records and 3,780 checked fields matched exactly, with zero mismatch.	Table 4.3
RQ3	Configurations H1–H4 required more logic and cycles than minimal configuration H0; configuration H4 added small timing changes relative to H3.	Tables 4.5 and 4.8
RQ4	Configuration H4 used all four profiles and the complete controller produced 45.455% fewer transitions than configuration H3 on the replay.	Table 4.6
RQ5	The held-out comparisons did not establish statistically significant route-quality improvement.	Table 4.7

4.19 Research-Hypothesis Outcomes

Table 4.13. Mapping from research hypotheses to evidence and outcomes.

Hypothesis	Evidence	Outcome	Interpretation
RH1	Five post-route maximum-frequency reports	Supported	Every configuration exceeded 100 MHz
RH2	3,780 field comparisons	Supported	Zero physical/reference mismatch
RH3	2.048 μ s H4 latency and implementation metrics	Supported	Microsecond-scale inference with measured resource/timing trade-offs
RH4	44 versus 24 transitions	Supported	Complete H4 policy-and-dwell controller switched less on the common replay
RH5	Paired CIs and p -values 0.2188 and 0.4531	Not supported	Route-quality superiority was not established

RH1–RH4 are supported by the reported evidence, whereas RH5 is not supported. The non-support of RH5 is a substantive experimental result: physical feasibility and adaptive behavior do not automatically imply statistically superior route quality.

4.20 Conclusion

Five independent FPGA configurations exceeded 100 MHz and matched 3,780 physical result fields exactly. Configuration H4 achieved 2.048 μ s mean kernel latency and exhibited a different, less frequently switching profile trace than configuration H3, with explicit timing and resource trade-offs. The held-out route-quality comparisons remained statistically neutral. Supporting VLSI results show a future physical-design path for isolated kernels without constituting an integrated ASIC.

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 Thesis Summary

This thesis developed and physically evaluated **QFlow**, a fixed-point FPGA accelerator for adaptive key-resource-aware route-candidate evaluation in trusted-relay QKD networks. The system accepts externally generated candidate descriptors, applies hard eligibility, selects a fixed or adaptive scoring profile, evaluates candidates with deterministic arithmetic, and returns a selected identifier and status. Candidate generation, topology management, KMS operation, SDN orchestration, security validation, and route installation remain control-plane functions.

The adaptive controller quantizes four features into a 256-state address and selects one of four scoring profiles. Tabular Q-learning is performed offline. The deployed reinforcement-learned configuration uses a 256-entry two-bit policy ROM and minimum-dwell control, and performs deterministic inference without runtime Q-value updates.

5.2 Main Findings

Five independent hardware evaluation configurations were implemented on a Digilent Nexys 3 Spartan-6 FPGA under a common interface, clock, replay, and measurement contract. All configurations exceeded the 100 MHz post-route target. The physical campaign produced 420 records and 3,780 checked fields, all of which matched the fixed-point reference exactly.

Configuration H4 achieved a post-route maximum frequency of 102.020 MHz and a mean normal-case accepted-request-to-done kernel latency of 2.048 μ s. Relative to configuration H3, configuration H4 changed mean latency by +0.986% and maximum frequency by -0.612% , mapped to 7.081% fewer LUTs, 5.378% more registers, and 4.961% more occupied slices. These values describe one implemented mapping and do not imply a universal resource property of reinforcement learning.

The complete configuration H4 controller, including the learned policy ROM and minimum-dwell mechanism, produced 24 profile transitions compared with 44 for configuration H3, a 45.455% reduction on the common replay. The corresponding route-quality comparisons were not statistically significant: the configuration H4 versus H2 and H4 versus H3 sign-test p -values were 0.2188 and 0.4531, and both confidence intervals crossed zero. The evidence therefore supports physical deployability, exactness, and distinct adaptive behavior, but does not establish route-quality superiority.

The supporting VLSI study showed that selected kernels can pass through generic synthesis and separate OpenROAD flows. Fixed-coefficient specialization reduced one scoring kernel from 5,343 to 503 generic cells, while isolated routed areas were 6,451 μm^2 for SKAG-W1,

22,186 μm^2 for FDPE-V3, and 3,368 μm^2 for Pareto-C0. These results indicate a possible future ASIC path; they are not an integrated ASIC implementation.

5.3 Contributions

The work contributes:

1. a classical key-resource model for candidate evaluation that represents pool occupancy, generation, consumption, status, utilization, health indicators, and policy-controlled freshness;
2. a bounded fixed-point evaluator with separate hard eligibility and soft multi-objective preference;
3. fixed, rule-adaptive, and offline-learned profile-selection mechanisms;
4. a deterministic policy-ROM deployment architecture with minimum-dwell control;
5. five independent FPGA configurations and a common physical measurement contract; and
6. reproducible exactness, timing, latency, resource, adaptive-behavior, and neutral route-quality evidence.

The principal contribution is thus a reproducible hardware-systems result rather than a claim of universal routing superiority.

5.4 Limitations

The candidate evaluator was tested on a bounded record format, one FPGA board and device family, and 84 records per configuration. The replay establishes exact implementation behavior for those inputs but does not establish network-level superiority on unseen topologies or live QKD deployments. The policy was trained in a controlled research environment, and replay rows may contain dependence that reduces the effective statistical sample size.

No exactly matched CPU/FPGA timing benchmark, activity-calibrated FPGA power measurement, or CPU/FPGA energy comparison was completed. No live KMS, SDN controller, or QKD device was integrated, and the host/FPGA link was an experimental transport rather than a production authenticated interface. The VLSI results concern separate kernels and do not provide integrated area, full-chip power, foundry signoff, tapeout, or fabricated performance.

5.5 Future Work

The highest-priority next step is an exactly matched CPU/FPGA benchmark using the same candidate descriptors, fixed-point contract, request protocol, and timing boundary. This would

support a defensible kernel speed comparison and expose the effect of software scheduling and data transfer.

Second, FPGA power and energy should be measured with activity-calibrated switching data and a defined operating point for every configuration. Energy per accepted candidate-evaluation transaction should be separated from board idle power and host transport.

Third, the policy should be tested on larger independent trace sets and unseen topologies. The evaluation should include stronger RL and non-RL baselines, bursts, key-pool depletion and recovery, failures, changing key-generation rates, and statistically powered route-quality analysis.

Fourth, a policy-by-dwell ablation should independently vary the policy and minimum-dwell mechanism. This would determine how much of the observed switching reduction arises from the learned mapping, the dwell rule, and their interaction.

Fifth, **QFlow** should be integrated with a live KMS/SDN testbed. The integration must measure metadata acquisition, candidate generation, host/FPGA communication, security-policy checks, fallback, and route installation separately from kernel latency.

Sixth, the host/FPGA interface should be authenticated and hardened. Input validation, replay protection, policy identity, signed bitstreams where supported, fault containment, audit logging, and software fallback are important for a security-sensitive controller.

Finally, an integrated full-core ASIC study can be considered after the mathematical and interface contracts stabilize. Such work would require complete logic and physical verification, clock and power design, timing closure, DRC/LVS, packaging, test planning, and a measured comparison with the FPGA implementation.

5.6 Closing Statement

The experiments show that an offline-learned key-resource-aware controller can be deployed bit-exactly on a resource-constrained FPGA with deterministic microsecond-scale candidate-evaluation latency and controlled implementation overhead. Equally important, the neutral route-quality result defines where the evidence stops. This combination of physical exactness, transparent trade-offs, and bounded interpretation provides a sound basis for larger network experiments and future deployment studies.

CHAPTER A

REPRODUCIBILITY AND ARTIFACT REGISTER

A.1 Repository snapshots

The project repository is available at <https://github.com/rizvi19/thesis-qcnfpga>. The primary FPGA and controller evidence is preserved on branch `rl-nexys3-adaptive` at commit `290b4ad75eef2af2b9da2f1f281a2b89416cfd24`. The supporting VLSI evidence is preserved on branch `vlsi-kernel-study` at commit `0093ba288f26a1520e4318f207e13f52d38db5a8`. The submission source branch is `thesis-final-submission-polish`; its commit is reported in the repository history and submission handoff.

A.2 Primary artifact register

Table A.1. Primary reproducibility artifacts.

Artifact	Repository location
RL model description	<code>docs/rl/rl_model_card.md</code>
Policy selection	<code>results/rl/p3_revision/selection_record.json</code>
Policy freeze	<code>results/rl/p3_policy_freeze/freeze_record.json</code>
Held-out controller results	<code>results/rl/p3_heldout/</code>
Controller verification	<code>results/rl/p4_exhaustive/</code> and <code>results/rl/p4_integrated/</code>
Integration checkpoint	<code>results/nexys3/p5_step9_final_checkpoint/</code>
Independent implementation reports	<code>results/nexys3/p6_step3_h0_ise/</code> through <code>results/nexys3/p6_step8_h4_ise/</code>
Physical records and comparisons	<code>results/nexys3/p6_step9_cross_mode/</code>
Submission-ready physical summaries	<code>results/nexys3/p6_step10_closure/</code>
VLSI kernel results	<code>results/partC_vlsi/</code>
CMOS primitive results	<code>results/partD_cmos/</code>
Literature verification	<code>literature/p1/audit/</code> and <code>literature/p1/corrected/</code>

A.3 Headline numerical sources

Table A.2. Authoritative sources for headline numerical results.

Reported quantity	Source artifact
Physical records, exact fields, configuration H4 summary, and H4/H3 percentages	results/nexys3/p6_step10_closure/final_summary.json
H0–H4 post-route timing, cycles, resources, and exactness	results/nexys3/p6_step10_closure/thesis_ready_same_board_table.csv
Profile counts and transitions	results/nexys3/p6_step10_closure/thesis_ready_adaptation_table.csv
Paired route-quality statistics	results/nexys3/p6_step10_closure/thesis_ready_pairwise_statistics.csv
Integration timing inventory	results/nexys3/p5_step9_final_checkpoint/p5_timing_inventory.csv
SKAG generic synthesis	asic/skag_weight_kernel/results/skag_yosys_comparison.csv
Other kernel generic synthesis	asic/fdpe_kernel/results/fdpe_yosys_comparison.csv and asic/pareto_cmp_kernel/results/pareto_yosys_comparison.csv
OpenROAD area and utilization	the three logs/6_report.log files below results/partC_vlsi/openroad_<kernel>/
Multiplexer delay and load calculation	results/partD_cmos/pareto_cmp_primitive/load_sweep/tg_mux_load_sweep.csv
Literature-search date, queries, and dispositions	literature/p1/audit/search_log.csv, paper_audit.csv, and literature/p1/corrected/PROVENANCE.md

A.4 Evidence classification

Table A.3. Evidence types used in the thesis.

Evidence type	Interpretation
Physical replay	Output captured from a programmed Nexys 3 FPGA
Post-route implementation	Timing and resources after placement and routing
RTL simulation	Cycle-accurate behavior of the register-transfer-level design
Software simulation	Reference-model, training, and network/controller output
Analytical result	Quantity calculated from a stated equation and inputs
Supporting physical design	OpenROAD result for an isolated kernel

A.5 Tool versions

The FPGA builds used Xilinx ISE 14.7. The final thesis was built with XeTeX 3.141592653-2.6-0.999995 and BibTeX 0.99d from MiKTeX 24.1. The retained OpenROAD reports identify OpenROAD v2.0-19993-ga613c7e53. Remaining Python, Icarus Verilog, Yosys, ngspice, and environment details are recorded by the version-controlled build scripts and logs associated with each artifact family.

A.6 Build and replay sequence

1. check out the branch and commit associated with the selected evidence set;
2. verify registered source, ROM, and replay hashes;
3. run the software reference and controller regression tests;
4. compile and run unit and integrated RTL simulations;
5. build one selected hardware configuration with Xilinx ISE 14.7;
6. inspect synthesis, map, placement-and-route, and timing reports;
7. program the volatile FPGA bitstream;
8. run the common UART replay and retain the raw response;
9. parse the response and compare all expected fields; and
10. generate cross-configuration tables only after every configuration passes exactness.

The CSV files under `data/` in the thesis package are direct copies or compact extracts of the registered repository summaries. LaTeX figures use those reported values or architectural diagrams drawn in the source.

CHAPTER B

FIXED-POINT, STATE, AND PROTOCOL CONTRACT

B.1 State and policy layout

Table B.1. Adaptive-controller storage layout.

Artifact	Depth	Useful width	Ordering
Policy ROM	256	2 bits	State IDs 0–255
Profile ROM	4	18 bits	Profiles Π_0 – Π_3
Training Q-table	1,024	16 bits	State-major, action-minor, signed Q8.7

The deployed controller reads the policy ROM and profile ROM. The training Q-table is retained for analysis but is not part of the inference datapath. The state address is

$$z = (((b_B \times 4) + b_R) \times 4 + b_L) \times 4 + b_I. \quad (\text{B.1})$$

B.2 Threshold definitions

For thresholds $\theta_1 < \theta_2 < \theta_3$, the four-level bin function is

$$b(x; \theta_1, \theta_2, \theta_3) = \begin{cases} 0, & x < \theta_1, \\ 1, & \theta_1 \leq x < \theta_2, \\ 2, & \theta_2 \leq x < \theta_3, \\ 3, & x \geq \theta_3. \end{cases} \quad (\text{B.2})$$

Equality enters the higher bin. Testing below, at, and above every threshold gives 36 directed threshold-edge cases for four features and three thresholds.

B.3 Profile payload

Each α coefficient is encoded as a three-bit unsigned half-unit numerator, and each Tchebycheff ratio uses two unsigned bits. The 18-bit word is

$$\alpha_1[2:0] \alpha_2[2:0] \alpha_3[2:0] \alpha_4[2:0] \lambda_1[1:0] \lambda_2[1:0] \lambda_3[1:0].$$

The four hexadecimal payloads are 13329, 1B516, 14399, and 0A2A5.

B.4 Request and result protocol

Table B.2. Synchronous kernel protocol.

Condition	Required behavior
Reset	Clear busy/done, controller history, and dwell state
Valid start while ready	Sample all inputs once and begin evaluation
Start while busy	Apply the declared busy-response status without resampling inputs
Invalid candidate fields	Return invalid status
No feasible candidate	Return no-path status
Back-to-back request	Accept at the first permitted ready edge
Done	Assert on the synchronous completion edge with stable result fields

The raw UART record carries the request identifier and the returned status, configuration/profile identity, state, selected candidate, score, and cycle-related fields required by the registered nine-field comparison. The UART layer transports the record but is not included in kernel-cycle measurement.

B.5 Latency boundary

Cycle count begins at synchronous request acceptance and ends at the synchronous done edge. Host parsing, UART serialization, operating-system scheduling, JTAG, programming, KMS access, candidate generation, and route installation are outside this boundary.

CHAPTER C

COMPLETE ADDITIONAL RESULTS

C.1 Additional Same-Board Route-Quality Results

Table C.1. Held-out status and route-quality summaries.

Configuration	Selection method	Rows	Success	No path	Invalid	Mean quality	Median quality
H0	feasible shortest distance	64	58	4	2	61707.19	62087.50
H1	fixed key-aware	64	58	4	2	61640.03	61673.50
H2	fixed-profile QFlow	64	58	4	2	61640.03	61673.50
H3	threshold/rule adaptive	64	58	4	2	61804.55	61673.50
H4	reinforcement-learned adaptive	64	58	4	2	61726.50	61837.00

C.2 Adaptive Profile Percentages

Table C.2. Adaptive profile use over 81 valid decisions.

Configuration	Π_0	Π_1	Π_2	Π_3
H3	29.630%	44.444%	22.222%	3.704%
H4	51.852%	13.580%	22.222%	12.346%

C.3 Controller Overhead Relative to Configuration H0

Table C.3. Absolute resource and cycle differences relative to minimal configuration H0.

Configuration	Δ registers	Δ LUTs	Δ slices	Δ normal cycles
H1	507	683	288	199.763
H2	592	849	348	200.763
H3	604	1,035	344	200.763
H4	663	905	376	202.763

These differences quantify functional additions relative to a deliberately minimal reference. They are not an isolated cost of reinforcement learning.

C.4 Integration Timing Inventory

Table C.4. Post-route timing checkpoints for successively integrated blocks.

Block	Minimum period (ns)	Maximum frequency (MHz)	Status
B1	6.253	159.923	Pass
B2	9.591	104.264	Pass
B3	6.517	153.445	Pass
B4	8.391	119.175	Pass
B5	9.670	103.413	Pass
B6	9.252	108.085	Pass

C.5 Supporting VLSI Detail

Table C.5. Generic Yosys synthesis results for isolated kernel variants.

Variant	Implementation distinction	Generic cells	Approx. error
FDPE-V0	256-entry floor lookup	2,258	3.1853%
FDPE-V1	128-entry floor lookup	1,944	6.4597%
FDPE-V2	128-entry linear interpolation	3,399	0.0587%
FDPE-V3	64-entry linear interpolation	3,014	0.1986%
SKAG-W0	Four runtime 16-by-16 products	5,343	—
SKAG-W1	Fixed-coefficient shift-add logic	503	—
Pareto-C0	Full dominance and tie-break	497	—
Pareto-C1	Score-first priority selector	472	—

The FDPE rows document a supplementary interpolation arithmetic study rather than current candidate-evaluator functionality. Approximation error is reported only for that isolated numerical experiment.

Table C.6. Simplified ngspice transmission-gate multiplexer load sweep.

Load (fF)	Average delay (ps)	Capacitive-load quantity (fJ)
5	10.38	8.10
10	17.27	16.20
20	26.64	32.40
50	54.31	81.00

The last column is the analytical $\frac{1}{2}CV^2$ value for the stated load at 1.8 V. It is not measured **QFlow** energy per route decision.

CHAPTER D

EXPERIMENTAL SCOPE AND REPRODUCIBILITY NOTES

D.1 Measurement boundaries

Physical-board evidence refers to records returned by a programmed Nexys 3 FPGA. Post-route timing and resource values come from Xilinx ISE implementation reports. RTL and software results are identified as simulations, and the OpenROAD results refer to isolated physical-design kernels. These evidence types answer different questions and are not combined into a single performance measure.

The reported kernel latency begins when a synchronous request is accepted and ends when the FPGA asserts `done`. It excludes host processing, UART serialization, JTAG programming, KMS acquisition, candidate generation, security checks, and route installation.

D.2 Repository milestone identifiers

Some repository directories retain P0–P6 prefixes because the evidence was archived in sequential engineering milestones. The directories identify the software reference and contract foundation, literature verification, policy training and freeze, controller verification, FPGA integration, and same-board replay stages described by their accompanying completion records. These identifiers are repository milestone labels and are not experimental variables. The scientific variables used in this thesis are the hardware evaluation configurations H0–H4, the research hypotheses RH1–RH5, scoring profiles Π_0 – Π_3 , and candidate routes $\mathcal{R}_i$.

D.3 Reproducibility records

Every headline result can be traced to a version-controlled CSV, JSON summary, implementation report, raw replay record, or physical-design report. Appendix A lists the relevant branches, commits, and paths. The standalone file `FINAL_THESIS_NUMERICAL_AUDIT.md` provides a compact value-by-value index for the submission package.

REFERENCES

- [1] Yuan Cao, Yongli Zhao, Qin Wang, Jie Zhang, Soon Xin Ng, and Lajos Hanzo. The evolution of quantum key distribution networks: On the road to the Qinternet. *IEEE Communications Surveys & Tutorials*, 24(2):839–894, 2022.
- [2] Miralem Mehic, Marcin Niemiec, Stefan Rass, Jiajun Ma, Momtchil Peev, Alejandro Aguado, Vicente Martin, Stefan Schauer, Andreas Poppe, Christoph Pacher, and Miroslav Voznak. Quantum key distribution: A networking perspective. *ACM Computing Surveys*, 53(5):1–41, 2020.
- [3] Vicente Martin, Alejandro Aguado, Juan P. Brito, David R. Lopez, Victor Lopez, Momtchil Peev, Christoph Pacher, Alberto Pastor, Juan P. B. Palomar, and Diego R. Lopez. MadQCI: A heterogeneous and scalable SDN-QKD network deployed in production facilities. *npj Quantum Information*, 10, 2024.
- [4] Hao-Ze Chen et al. Implementation of carrier-grade quantum communication networks over 10000 km. *npj Quantum Information*, 11, 2025.
- [5] Lin Bi, Minghui Miao, and Xiaoqiang Di. A dynamic-routing algorithm based on a virtual quantum key distribution network. *Applied Sciences*, 13(15), 2023.
- [6] Evgeniy O. Kiktenko, Andrey Tayduganov, and Aleksey K. Fedorov. Routing algorithm within the multiple non-overlapping paths’ approach for quantum key distribution networks. *Entropy*, 26(12), 2024.
- [7] Qiaolun Zhang, Omran Ayoub, Alberto Gatto, Jun Wu, Francesco Musumeci, and Massimo Tornatore. Routing, channel, key-rate, and time-slot assignment for QKD in optical networks. *IEEE Transactions on Network and Service Management*, 21(1):148–160, 2024.
- [8] Yuanchen Hao, Yuheng Xie, Wenpeng Gao, and Jianjun Tang. Q-learning for resource-aware and adaptive routing in trusted-relay QKD network. *Photonics*, 12(10), 2025.
- [9] Tim Johann, Sebastian Kühn, and Stephan Pachnicke. Adaptive routing for meshed QKD networks of flexible size using deep reinforcement learning. *Photonics*, 13(2), 2026.
- [10] Qing Lu, Zhenguo Lu, Hongzhao Yang, Shen-Shen Yang, and Yongmin Li. FPGA-based implementation of multidimensional reconciliation encoding in quantum key distribution. *Entropy*, 25(1), 2023.
- [11] Natarajan Venkatachalam, Foram P. Shingala, C. Selvagangai, S. Hema Priya, S. Dillibabu, Pooja Chandravanshi, and Ravindra P. Singh. Scalable QKD postprocessing system with reconfigurable hardware accelerator. *IEEE Transactions on Quantum Engineering*, 4:1–14, 2023.

- [12] Lianye Liao, Xinyi Wu, Ye Chen, Xiaodong Fan, Zhiyu Tian, Jinqun Huang, Tonglin Mu, Junran Guo, Minjie Liu, Bo Liu, and Shihai Sun. Efficient FPGA implementation of polar codes-based information reconciliation for quantum key distribution. *Scientific Reports*, 15, 2025.
- [13] Nishanth Chandra and Pradeep Kumar Krishnamurthy. FPGA-based synchronization of frequency-domain interferometer for QKD. *IEEE Transactions on Quantum Engineering*, 6:1–10, 2025. IEEE document 10769019.
- [14] Qingfu Zhang and Hui Li. MOEA/D: A multiobjective evolutionary algorithm based on decomposition. *IEEE Transactions on Evolutionary Computation*, 11(6):712–731, 2007.
- [15] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and T. Meyarivan. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2):182–197, 2002.
- [16] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3–4):279–292, 1992.
- [17] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, USA, 2 edition, 2018.
- [18] Li-Quan Chen, Jing-Qi Chen, Qian-Ye Chen, and Yong-Li Zhao. A quantum key distribution routing scheme for hybrid-trusted QKD network system. *Quantum Information Processing*, 22, 2023.
- [19] Md Shahbaz Akhtar, Krishnakumar G., Vishnu B., and Abhishek Sinha. Fast and secure routing algorithms for quantum key distribution networks. *IEEE/ACM Transactions on Networking*, 31(5):2281–2296, 2023.
- [20] Purva Sharma, Shubham Gupta, Vimal Bhatia, and Shashi Prakash. Deep-reinforcement-learning-based routing for quantum key distribution networks. *IET Quantum Communication*, 4:136–145, 2023.
- [21] Kaijie Wei, Devanshu Garg, Ryutaro Nagai, Takao Tomono, and Hideharu Amano. FPT-EMS: An FPGA implementation using NB-LDPC code for continuous-variable quantum key distribution. In *Proceedings of the 15th International Symposium on Highly Efficient Accelerators and Reconfigurable Technologies*, pages 117–125, 2025.
- [22] Barry Shackelford et al. A high-performance, pipelined, FPGA-based genetic algorithm machine. *Genetic Programming and Evolvable Machines*, 2(1):33–60, 2001.
- [23] W. Tang and L. Yip. Hardware implementation of genetic algorithms using FPGA. In *Proceedings of the 47th Midwest Symposium on Circuits and Systems*, volume 1, pages I–549–I–552, 2004.

- [24] Pradeep R. Fernando, Srinivas Katkoori, Didier Keymeulen, Ricardo Zebulum, and Adrian Stoica. Customizable FPGA intellectual property core implementation of a general-purpose genetic algorithm engine. *IEEE Transactions on Evolutionary Computation*, 14(1):133–149, 2010.
- [25] Liucheng Guo, David B. Thomas, and Wayne Luk. Automated framework for general-purpose genetic algorithms in FPGAs. In *Applications of Evolutionary Computation*, volume 8602 of *Lecture Notes in Computer Science*, pages 714–725. Springer, 2014.
- [26] Murat Peker. A fully customizable hardware implementation for general purpose genetic algorithms. *Applied Soft Computing*, 62:1066–1076, 2018.